Crash Recovery

Ioana Manolescu
Inria Saclay and Ecole polytechnique



Outline

- Crash recovery overview
- Write-ahead logging (WAL)
- The ARIES recovery model
- More on durability: replication

Data loss and database failure



"Three things are certain:
Death, taxes, and lost data.
Guess which has occurred."

Database Failures are Costly

TWELVE YEARS of US Air Force complaints lost in database crash

Backups never fail until you REALLY need them



STATE OF CONFUSION —

State Department database crash strands hundreds at the border

Outage caused by corrupted data, which was mirrored to backup—making it useless.

SEAN GALLAGHER - 6/23/2015, 8:14 PM

Thanks to the failure of a system used to collect and transmit fingerprints and photos, the US State Department has been unable to issue visas to travelers or guest workers for the past two weeks, *The New York Times* reports. While some of the systems related to visa processing have

any travelers from

rdware failure. In an e-

Amazon's Cloud Crash Disaster Permanently Destroyed Many Customers' Data



Henry Blodget

Apr. 28, 2011, 7:10 AM 108,398

FACEBOOK LINKEDIN TWITTER

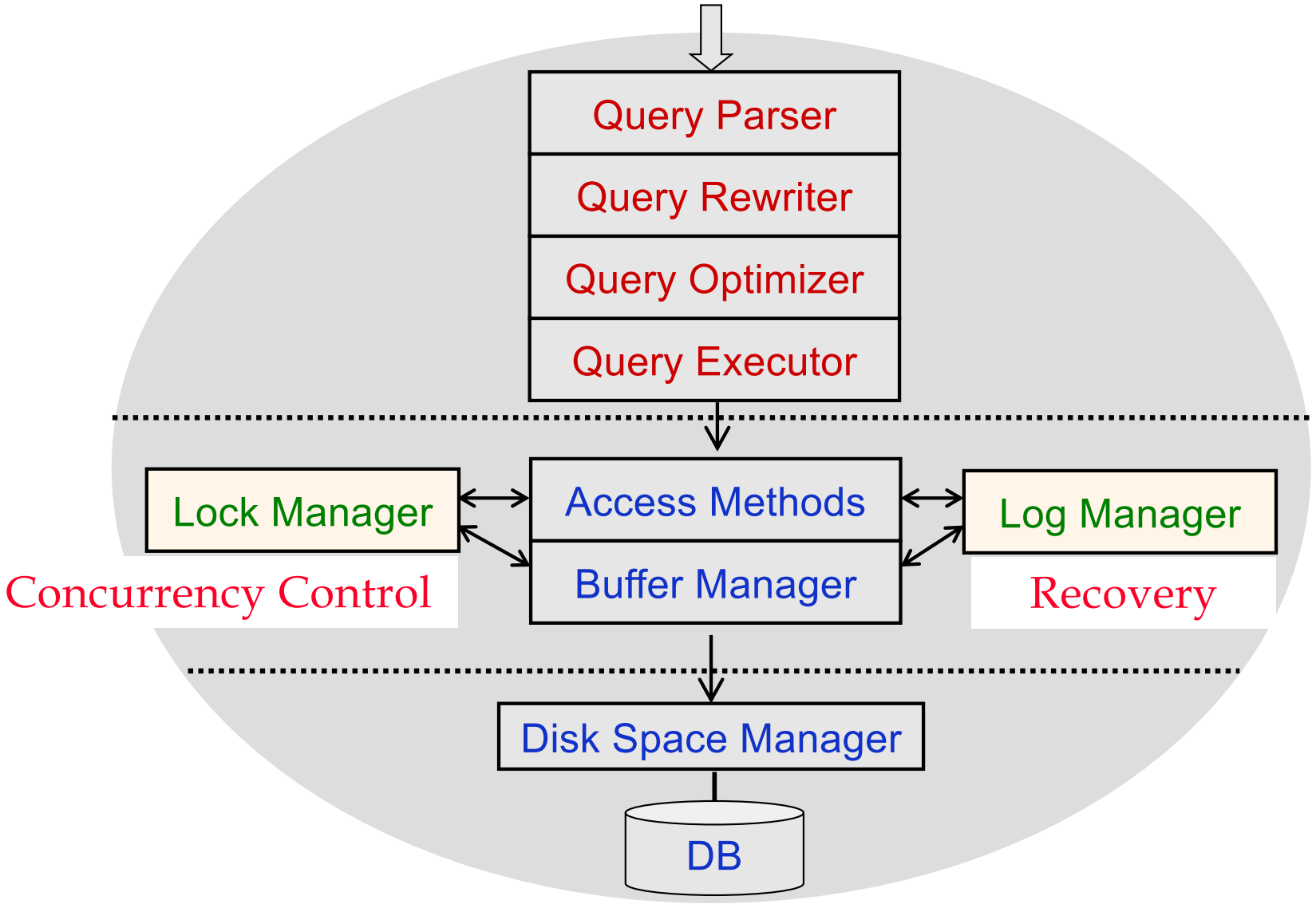
Amazon AWS Outage Shows Data in the Cloud is Not Always Safe

By Lawrence Abrams September 5, 2019 12:01 PM 13



A recent power outage outage at an Amazon AWS data facility and the resulting data loss for some customers shows that storing data in the cloud does not mean you do not also need a backup.

DBMS Architecture



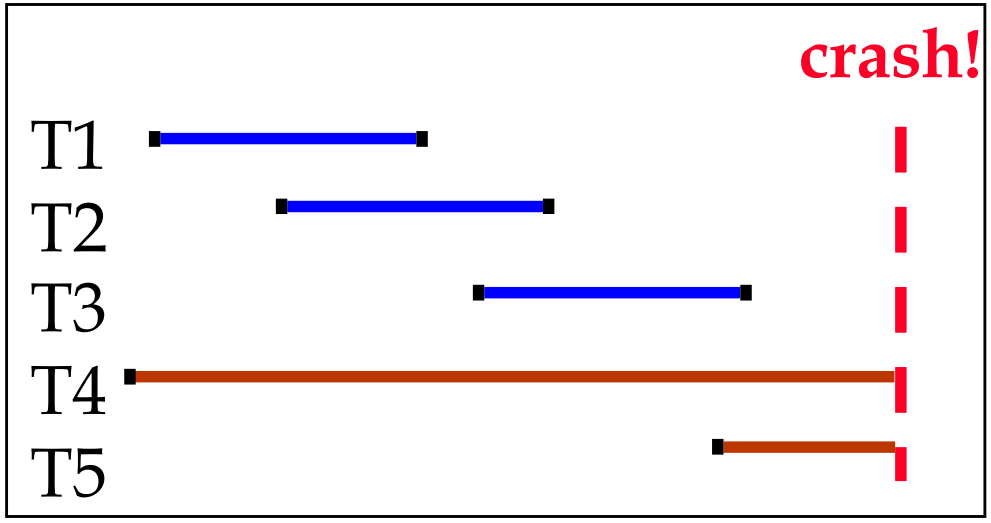
Scenarios involving Recovery

- What can happen?
  - A xact may abort because: the user cancels it, concurrency control kills it to avoid deadlock or take its lock...
  - The system may crash while transactions are in various stages of completion.
- The recovery manager guarantees Atomicity and Durability.
- **Atomicity**: *all-or-none* (aborted transaction effects should be rolled back)
- **Durability**: Effects of committed xacts should survive crashes.

Scenarios involving Recovery

- The recovery manager guarantees Atomicity and Durability.
  - **Atomicity**: *all-or-none* (aborted transaction effects should be rolled back)
  - **Durability**: Effects of committed xacts should survive crashes.

- Desired behavior after system restarts:
  - T1, T2 & T3 should be durable.
  - T4 & T5 should be aborted (effects not seen).



Assumptions

- Concurrency control is in effect.
  - For instance (usually) strict 2 Phase Locking.
  - Simultaneous reads and writes of the same object are handled properly.
- Any read, write or update happens on pages in memory.
- Updates are happening in place.
  - i.e. updates of an object will be written from memory to the only copy of it of on disk.

A naive scheme for Atomicity and Durability

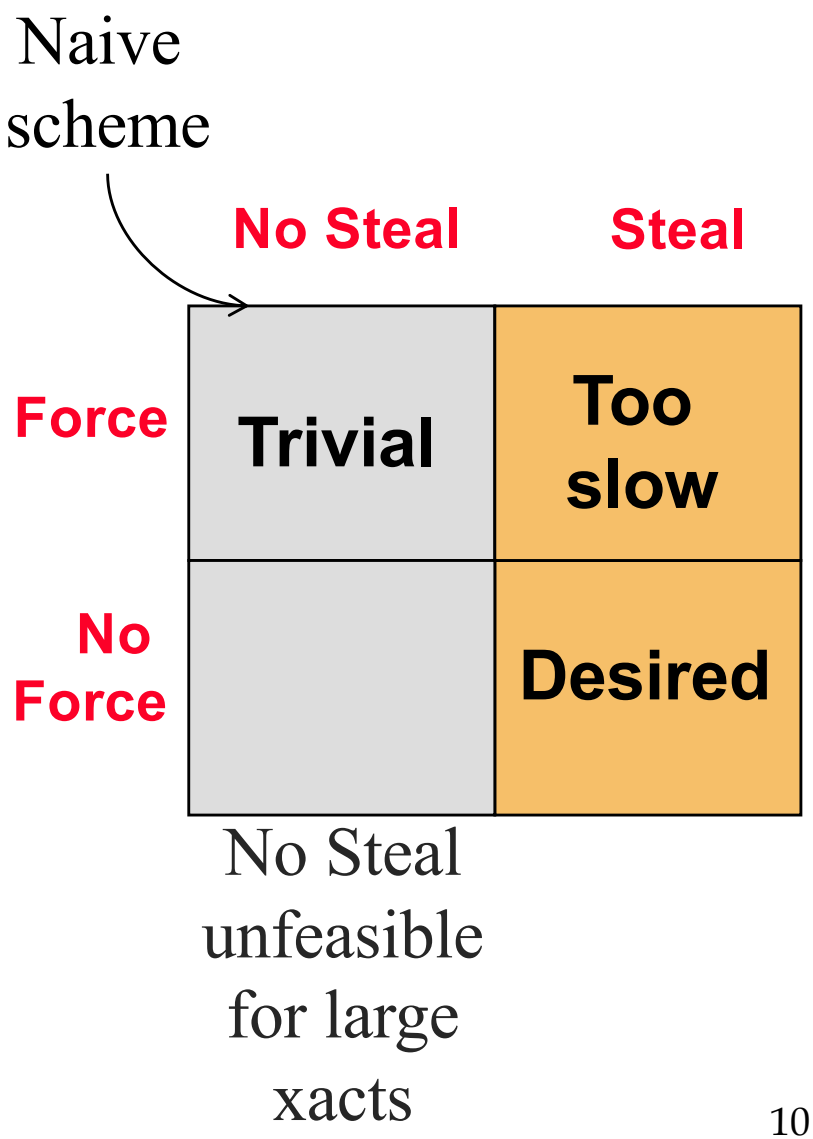
1. **All the updates of a xact only happen in the memory buffer pages** of the xact
2. The changes made by a xact are **written ("forced") to disk immediately when the xact commits**

However:

- This entails that every xact keeps all its modified pages in the buffer pool for as long as it runs → a xact accessing more pages than the buffer size can never run
  - In practice, some dirty memory pages are *written to disk while xact T1 is still running*, to release the memory pages for another xact T2. We say T2 **steal**s memory pages from T1.
- If a highly used page is updated by 20 xacts in memory, force would always entail 20 I/Os → costly!

Force, Steal, Performance and Recovery

- Force every write to disk at commit time?
  - Provides durability, but poor response time.
  - No force, how can we ensure durability?
- Steal buffer-pool frames from uncommitted Xacts?
  - If not, poor throughput.
  - If steal, how can we ensure atomicity?



Outline

- Crash recovery overview
- Write-ahead logging (WAL)
- The ARIES recovery model
- More on durability: replication

Basic Idea: Logging

- Log: a history of actions executed by DBMS
- Etymology:
  - From navigation (ship logs) which recorded ship speed
  - Earliest speed devices based on floating wood, counting knots



- Records for REDO/UNDO information for every update

  <XID, pageID, offset, length, old data, new data>

  - *Physical logging*: the bytes changed, shown above
  - *Logical logging*: the update operation itself (e.g., a query)
  - All commits and aborts
  - And additional control info (to be seen soon).

Basic Idea: Logging



- Log: a history of actions executed by DBMS
  - Records for REDO/UNDO information for every update

    <XID, pageID, offset, length, old data, new data>
    - *Physical logging*: the bytes changed, shown above
    - *Logical logging*: the update operation itself (e.g., a query)
  - All commits and aborts
  - And additional control info (to be seen soon).

From now on, time is materialized by LSNs

- Writing *log records* to disk is faster than writing *data pages*
  - *Sequential writes* to log (put it on a separate disk).
  - *Minimal info* written to log, often smaller than a data record; multiple updates fit in a single log page.

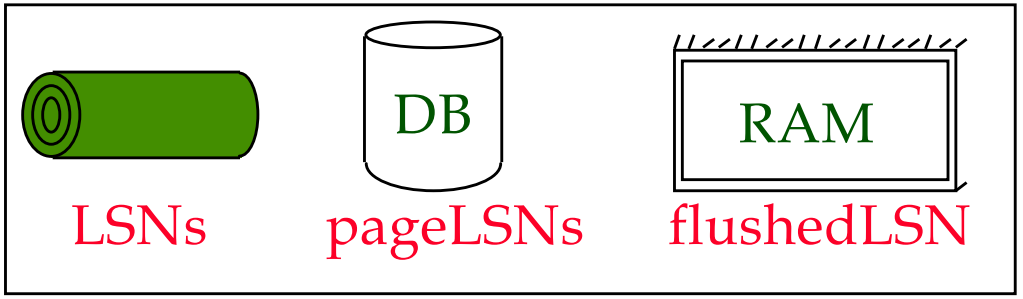
Write-Ahead Logging (WAL)

- Write-Ahead Logging (WAL) Protocol:
  1. Must force the **log record** for an update *before* the corresponding **data page** gets to disk (when steal).
     - Guarantees Atomicity
  2. Must write **all log records** for a Xact *before commit*.
     - Guarantees Durability

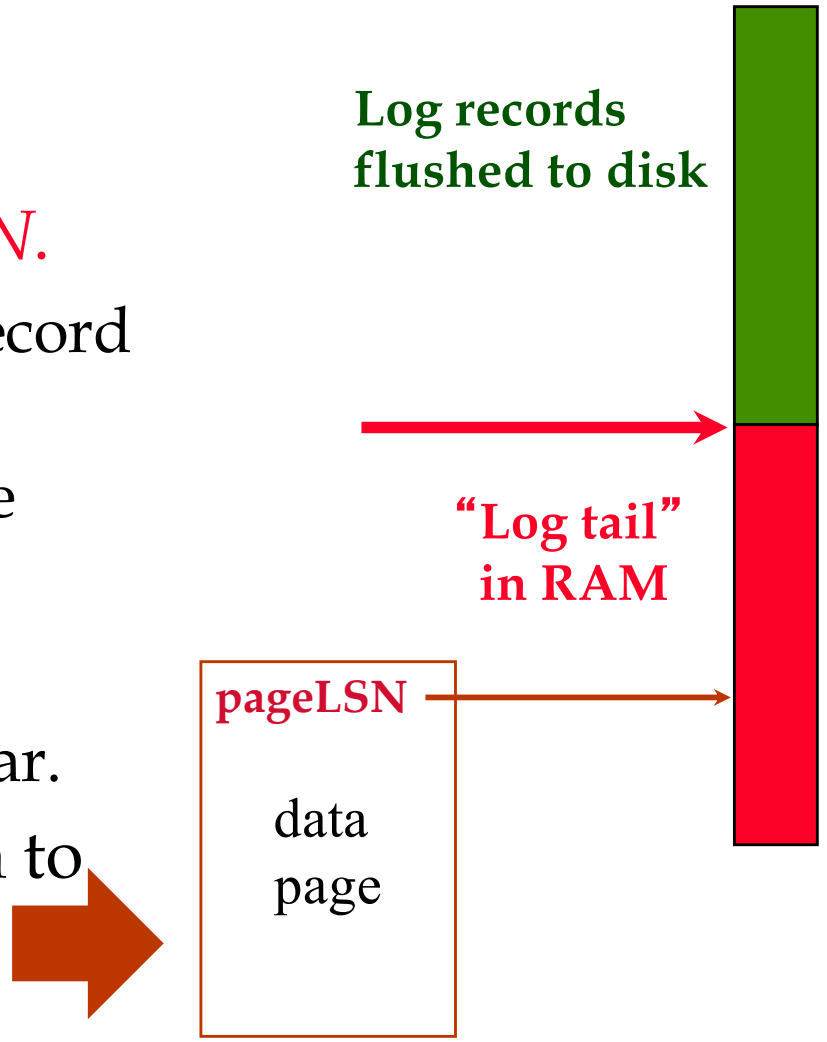
- Decouples Atomicity and Durability mechanisms from the actual saving of modified pages to disk!
  - The latter decided for performance: allow high throughput, smart replacement policies

- Exactly how is logging (and recovery) done?
  - We'll study the **ARIES** algorithm.

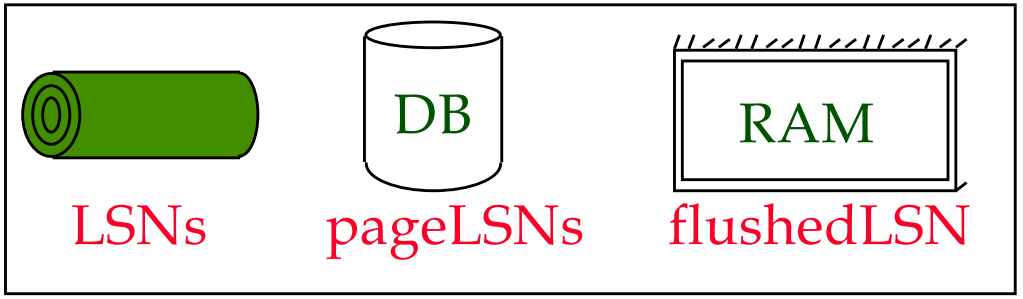
ARIES concepts



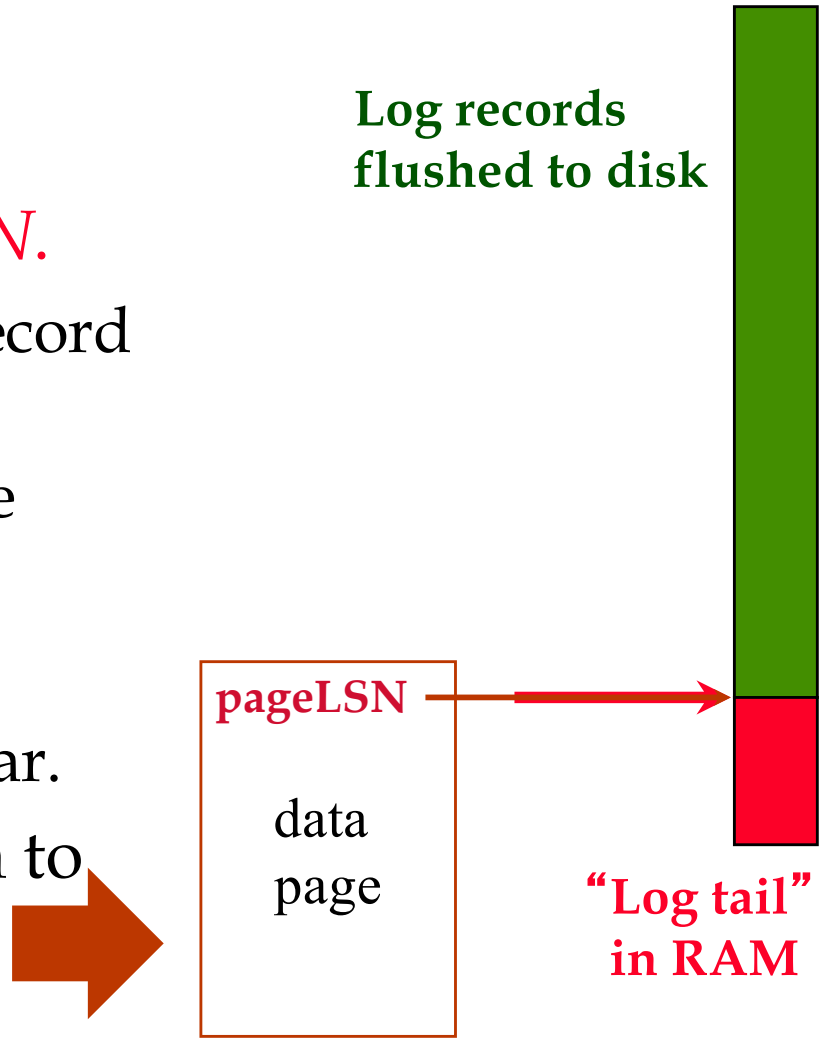
- Each log record has a unique *Log Sequence Number* (LSN).
  - LSNs always increasing.
- Each data page contains a *pageLSN*.
  - The LSN of the **most recent** log record for an **update** to that page.
  - When pageLSN changes, the page is in memory.
- System keeps track of *flushedLSN*:
  - The **max LSN** written to disk so far.
- WAL (1): Before a page is written to disk, flush (up to) its log record:
  - pageLSN ≤ flushedLSN



ARIES concepts

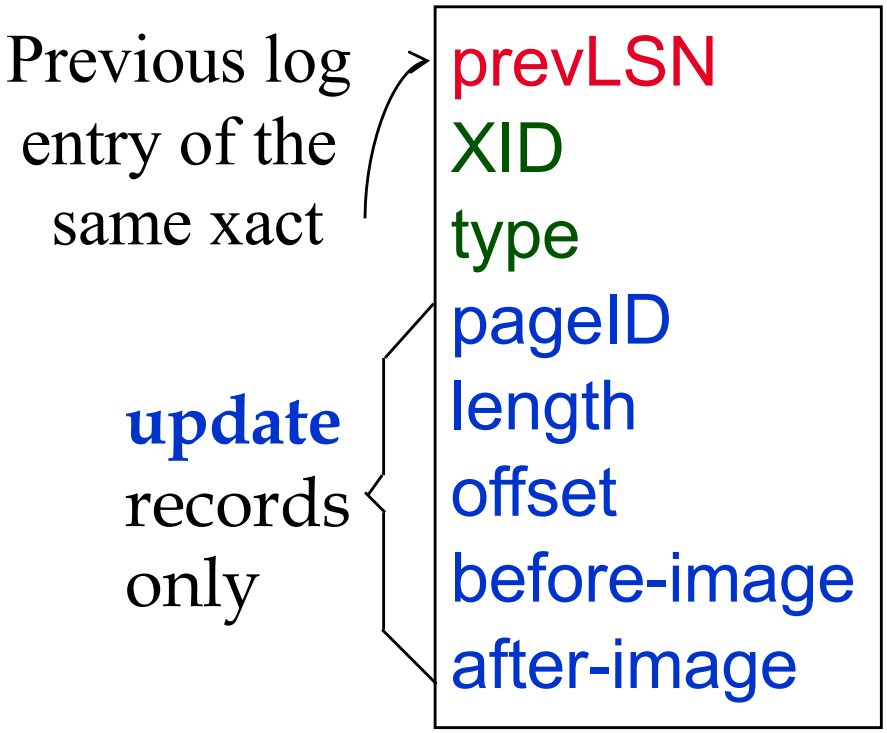


- Each log record has a unique *Log Sequence Number* (LSN).
  - LSNs always increasing.
- Each data page contains a *pageLSN*.
  - The LSN of the **most recent** log record for an **update** to that page.
  - When pageLSN changes, the page is in memory.
- System keeps track of *flushedLSN*:
  - The **max LSN** written to disk so far.
- WAL (1): Before a page is written to disk, flush (up to) its log record:
  - pageLSN ≤ flushedLSN



Log Records

LogRecord fields:



Possible log record types:

- **Update**
- **Commit**
- **Abort**
- **End** (signifies end of commit or abort)
- Compensation Log Records (CLRs)
  - for UNDO actions
- Begin/end checkpoint

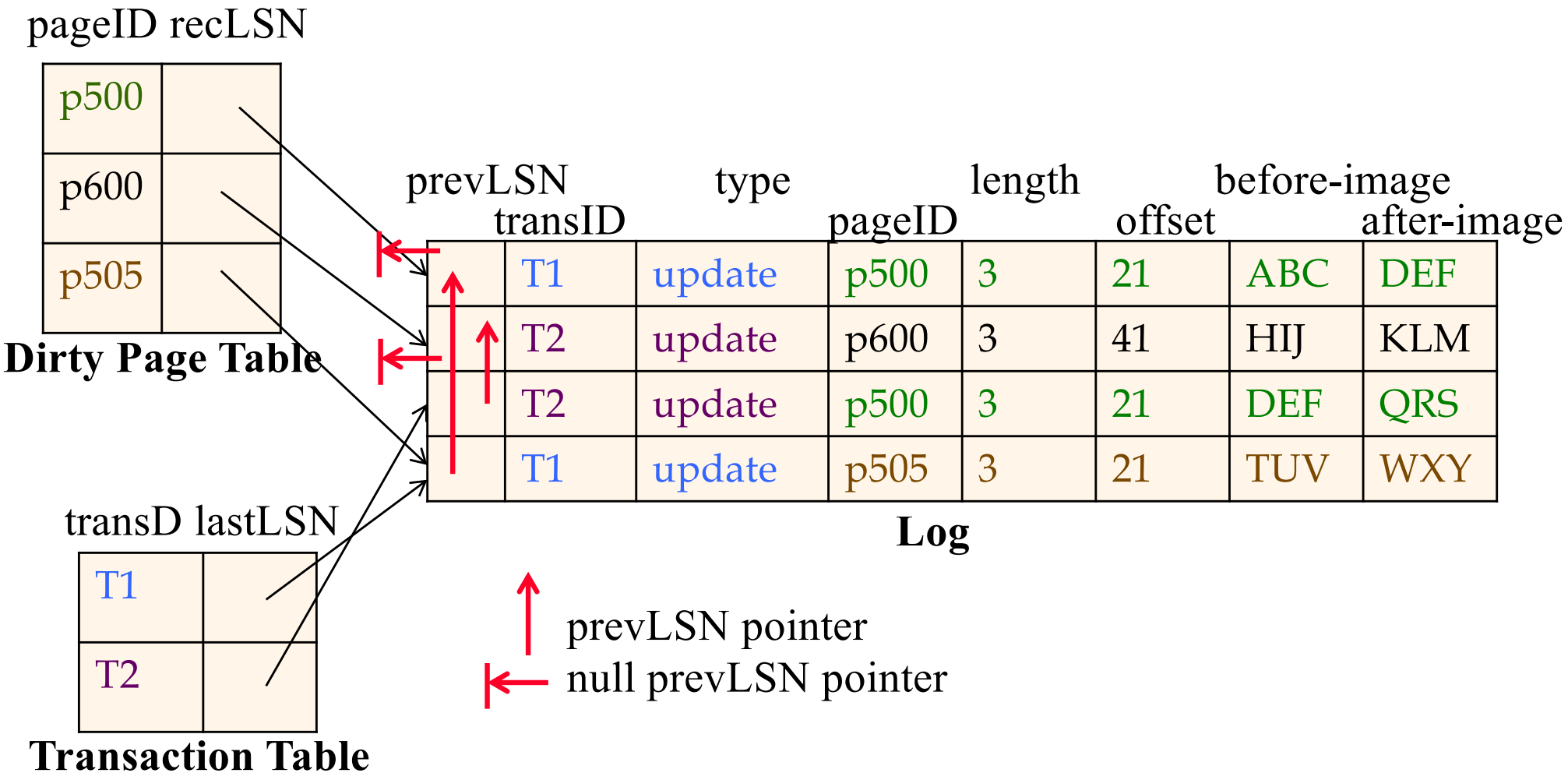
The log entries of *each xact* form a *backward-chained list*.

Other Log-Related Data Structures

- **Transaction Table**:
  - One entry per active *xact*.
  - Contains XID, status (running/committed/aborted), and lastLSN (of the **last/most recent** log record for xact)

- **Dirty Page Table**:
  - One entry per *dirty page* in buffer pool.
  - Contains recLSN (i.e., **recovery** LSN):
    the LSN of the log record which **first** caused the page to be dirty,
    or, equivalently:
    the LSN of the **oldest** update to this page, not yet written to disk.

Example


pageID recLSN
p500
p600
p505
Dirty Page Table
prevLSN
transID
type
pageID
length
offset
before-image
after-image
T1 update p500 3 21 ABC DEF
T2 update p600 3 41 HIJ KLM
T2 update p500 3 21 DEF QRS
T1 update p505 3 21 TUV WXY
Log
transD lastLSN
T1
T2
Transaction Table
prevLSN pointer
null prevLSN pointer

Big Picture: What is Stored Where



Data pages

each with a pageLSN



Log Records

prevLSN
XID
type
pageID
length
offset
before-image
after-image



Transaction Table

lastLSN (last update in Xact)
status

Dirty Page Table

recLSN (oldest page update not written to disk)

flushedLSN



Transaction Commit

- When a Xact is running:
  - Write to the log a record with LSN for each update operation.
  - Update Xact table and Dirty Page table in memory.
- Upon commit:
  1. Write *commit* record to log.
  2. WAL (2): All log records up to Xact's lastLSN are flushed.
     - Guarantees that flushedLSN ≥ lastLSN.
     - Log writes are sequential; many log records per log page.
  3. Then, write *end* record to log.

Checkpointing

- Periodically, DBMS creates a *checkpoint*, a snapshot of active xacts & dirty data", to minimize the time taken to recover from a system crash.

1. Write to log:
   - *begin_checkpoint* record: indicates when chkpt began.
   - *end_checkpoint* record: contains *Xact table* and *dirty page table* as of the time of begin_checkpoint
   - Force these to stable storage.
2. Store LSN of begin_checkpoint record (*master* record) in a safe place on stable storage.

- For simplicity, assume *quiescent* checkpointing:
  - Complete existing transactions, hold all incoming xacts.
- In practice, ARIES uses *fuzzy* checkpointing:
  - DBMS continues running as usual while end_checkpoint record is built, the records are flushed, etc.

Big Picture: What is Stored Where



Data pages

each with a pageLSN

master record

LSN of begin_chkpt



LogRecords

prevLSN
XID
type
pageID
length
offset
before-image
after-image



Transaction Table

lastLSN (last update in Xact)
status

Dirty Page Table

recLSN (oldest page update not written to disk)

flushedLSN



flushed In RAM

Outline

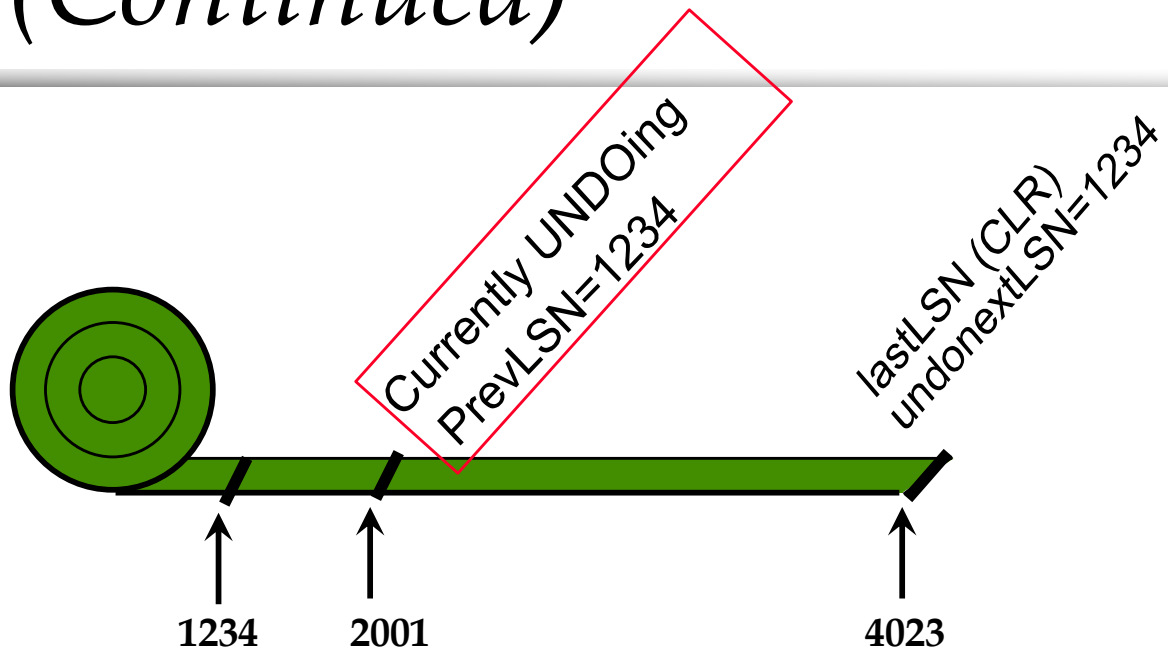
- Crash recovery overview
- Write-ahead logging (WAL)
- The ARIES recovery model
- More on durability: replication

Simple Transaction Abort (Rollback)

- For now, consider an *explicit abort* of a xact (no crash).
- **Goal**: undo all its actions.
- **ARIES algorithm** based on log and WAL:
  - "Play back" the log in reverse order, UNDOing updates:
  - Get lastLSN of xact from xact table.
  - Follow chain of log records backward via prevLSN field.
    - Replace after_image with before_image
    - To perform UNDO, also need to have a lock on data.
- Logging continues during UNDOs!

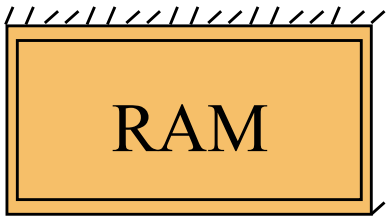
1. Before starting UNDO, write an *Abort* log record.
   - For recovering from *crash during UNDO*!

Abort (Continued)



2. Before restoring old value of a page, write a *Compensation Log Record* (CLR):
   - CLR has one extra field, undonextLSN, pointing to the next LSN to undo (i.e. the prevLSN of the record we're undoing now).
   - In the example, before the CLR, lastLSN of aborted xact=2001, which has prevLSN=1234;
     the CLR at 4023 becomes the xact's lastLSN
3. At end of UNDO, write an *end* log record.

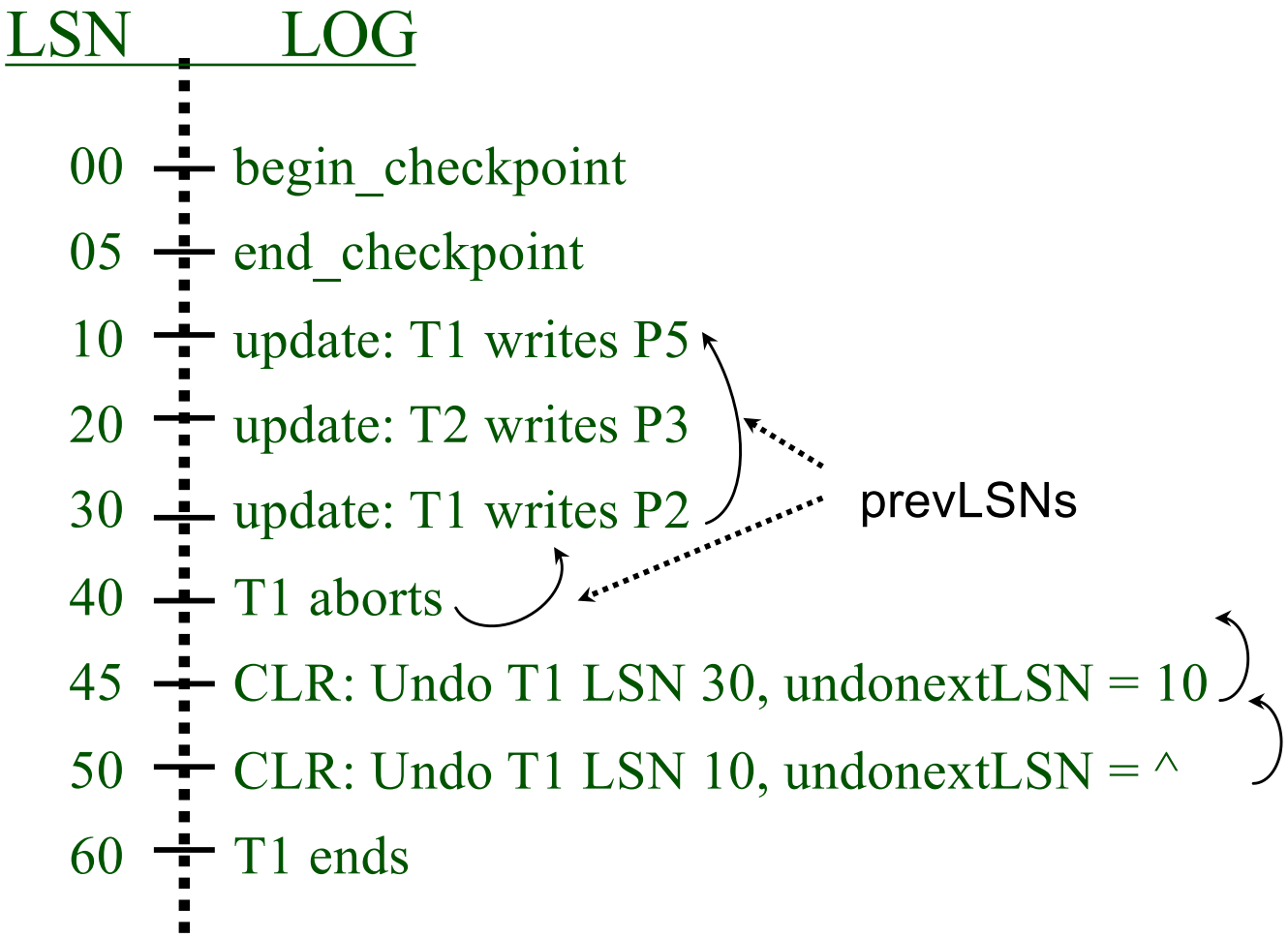
Example of Rollback



Xact Table (TT)
XID
lastLSN
status

Dirty Page Table (DPT)
recLSN (earliest)

flushedLSN



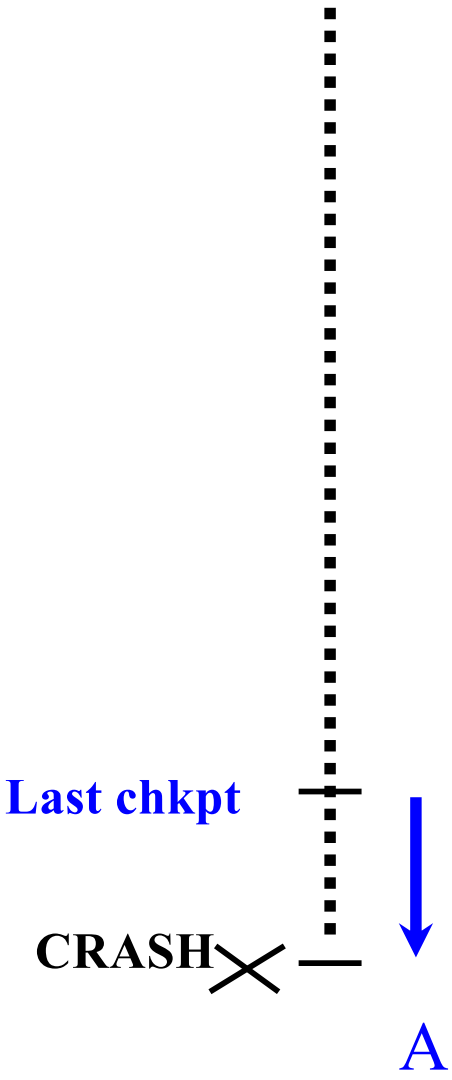
Crash recovery

- Crash: failure of the database server process
  - E.g., pg_ctl stop, or computer reboot
  - **Persistent** data is saved:
    - data, including page LSNs, and the master record
    - the log already flushed
  - **In-memory data** is lost
    - the log tail
    - the Xact table
    - the Dirty Page table
- Goal:
  - undo all xacts that did not finish committing
  - ensure all effects of committed xacts are saved

Main Principles behind ARIES

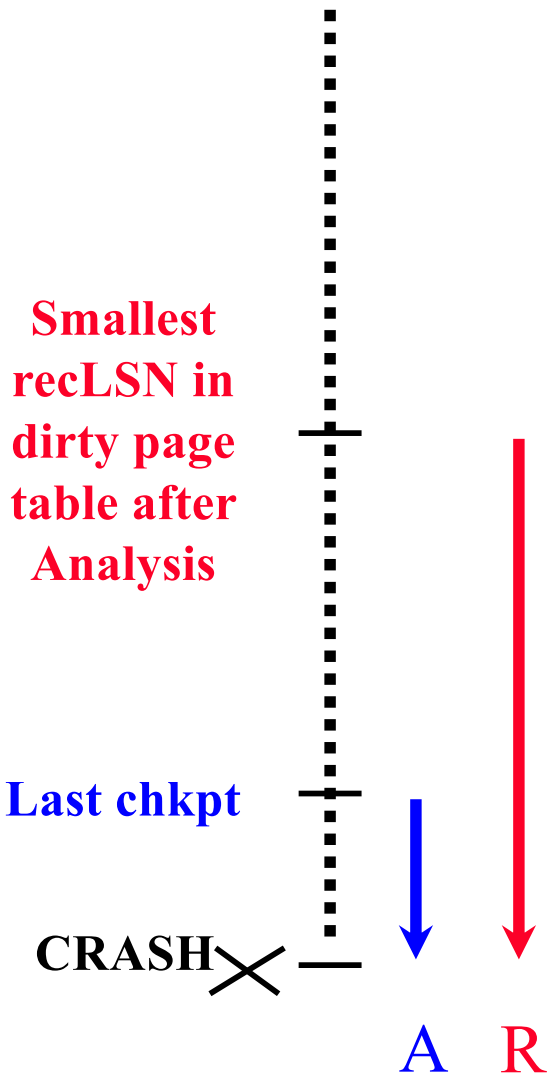
- Write-ahead logging (WAL)
  - Any change to an object is recorded in the log. The log is written to disk *before* the change to the object is written on disk.
- Repeating history during REDO
  - On restart after a crash, repeat all actions before the crash, brings the system as close as possible to the state that it was in before the crash.
  - Then undo the xacts still active at crash time.
- Logging changes during UNDO
  - Changes made to DB while undoing are logged to ensure such an action isn't repeated in the event of repeated crashes & restarts.

Big Picture of ARIES crash recovery



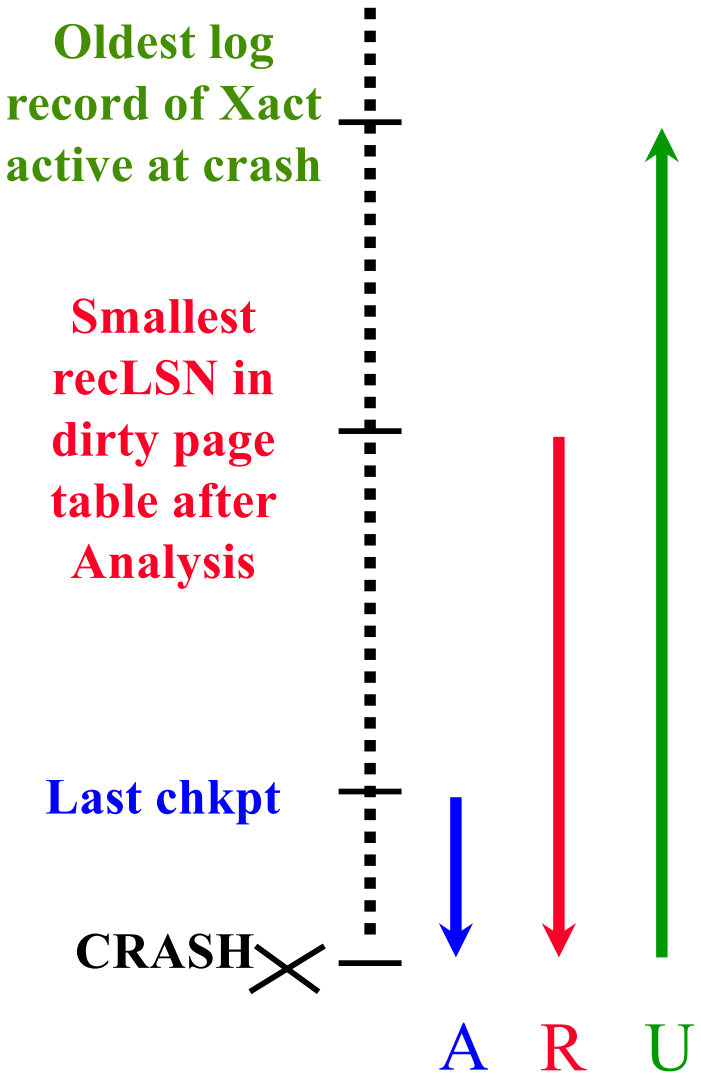
- Analysis figures which xacts had committed or failed at crash time.
  - Start from most recent checkpoint (found via master record).
  - Construct **Xact** and **Dirty Page** tables

Big Picture of ARIES crash recovery



- Analysis figures which xacts had committed or failed at crash time.
  - Start from most recent checkpoint (found via master record).
  - Construct **Xact** and **Dirty Page** tables
- REDO all actions; repeat history
  - Start from smallest recLSN in Dirty Page Table.

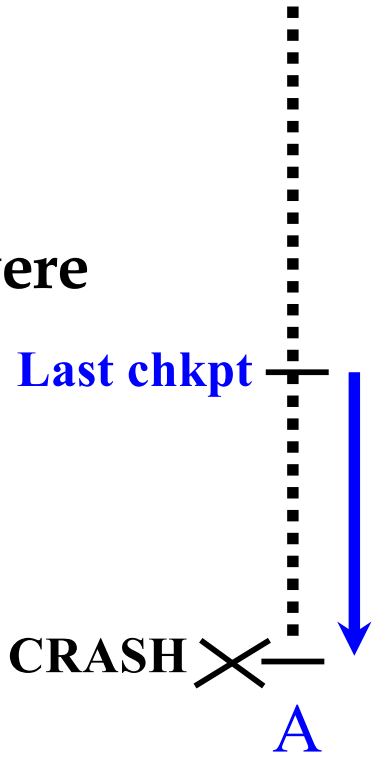
Big Picture of ARIES crash recovery



- Analysis figures which xacts had committed or failed at crash time..
  - Start from most recent checkpoint (found via master record).
  - Construct Xact and Dirty Page tables
- REDO all actions; repeat history
  - Start from smallest recLSN in Dirty Page Table.
- UNDO effects of failed Xacts.
  - Back to oldest LSN of a running xact at crash.

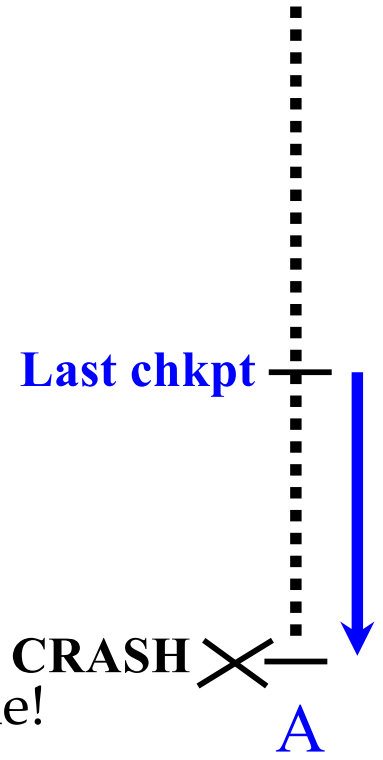
(1) *Recovery: The Analysis Phase*

- Goals:
  1. Find out **where to start the Redo**
  2. Find (a conservative superset of) **the pages that were dirty** at the time of the crash
  3. Identify xacts active at the time of the crash, which must be **undone**
- First, reconstruct *state at checkpoint*.
  - Get *begin_checkpoint* record via the *master* record.
  - Find its *end_checkpoint* record, read the xact table and dirty page table (DPT).
    - These reflect a state at begin_checkpoint, but more things may have happened between that time and the crash!



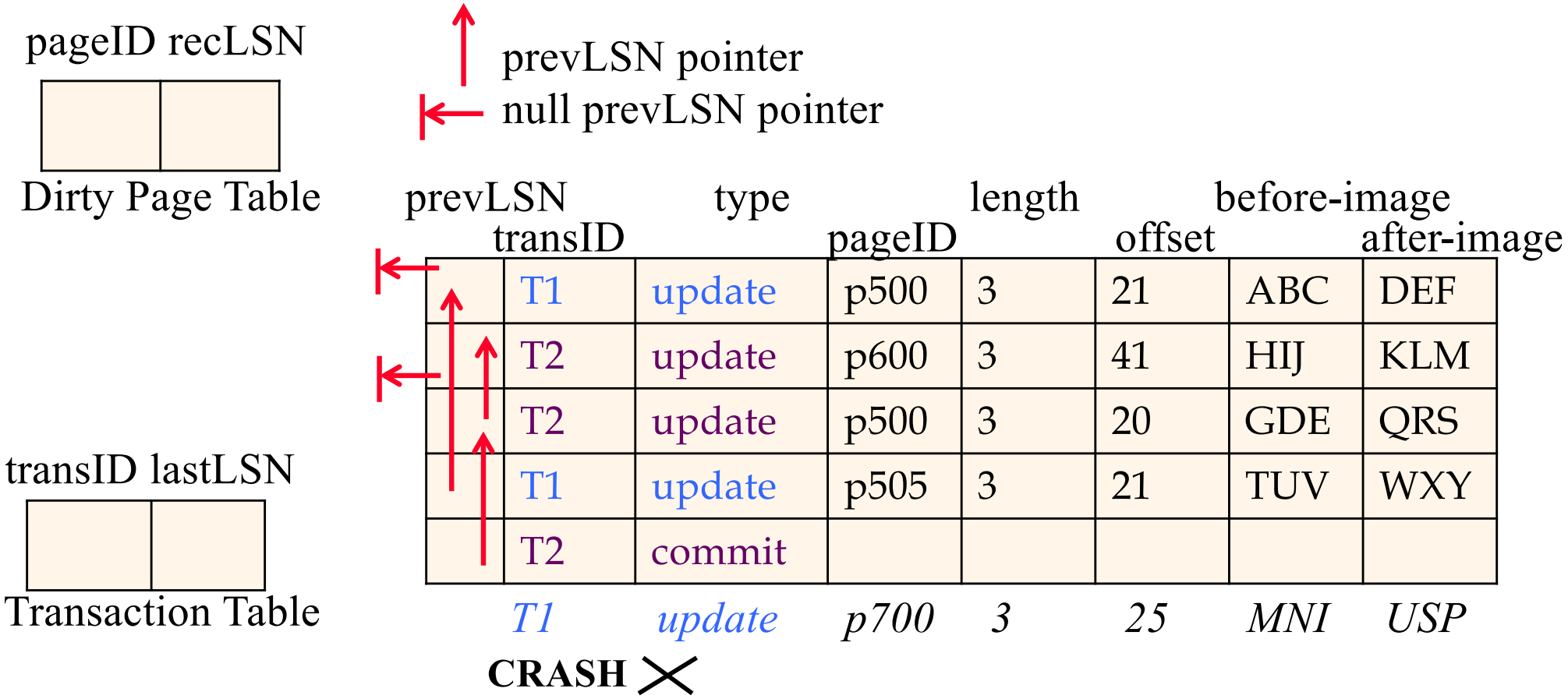
(1) *Recovery: The Analysis Phase*

- To reconstruct Xact Table and Dirty Page Table at crash, scan log forward (from last checkpoint):
  - *End* record for T → remove T from Xact table.
  - Other record for T:
    - If T is not in the Xact Table, add it.
    - Set lastLSN of T to the LSN of this record
    - If *commit* record, set T status to **C**.
  - A xact without C has status **U**: will have to be undone!
  - Upon *redoable* record (update or CLR) affecting page P:
    - If P not in Dirty Page Table, add it, and set its recLSN (earliest/oldest update) to the LSN of this record



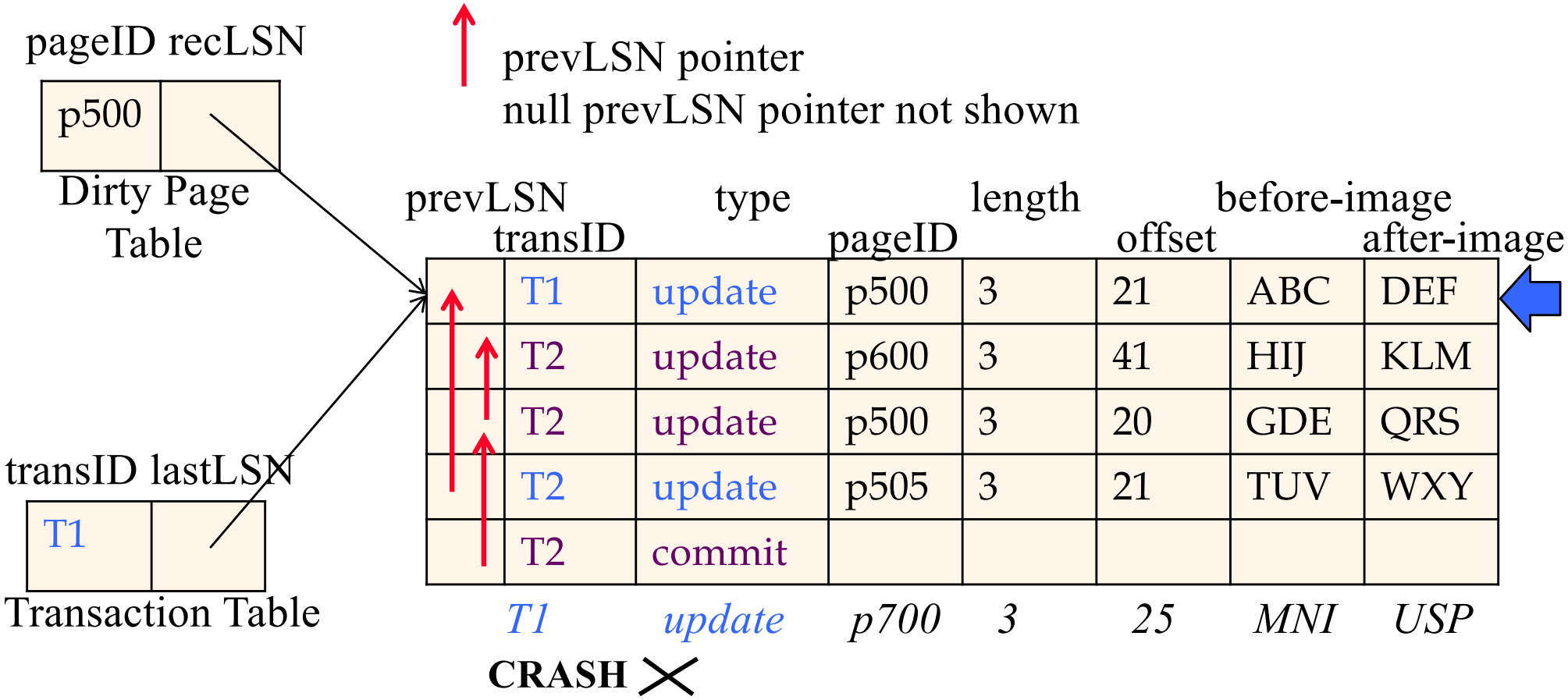
After Analysis, we have: the list of all transactions to undo (those with status U); the pages dirty at crash (+ maybe more)

Example: Analysis Phase



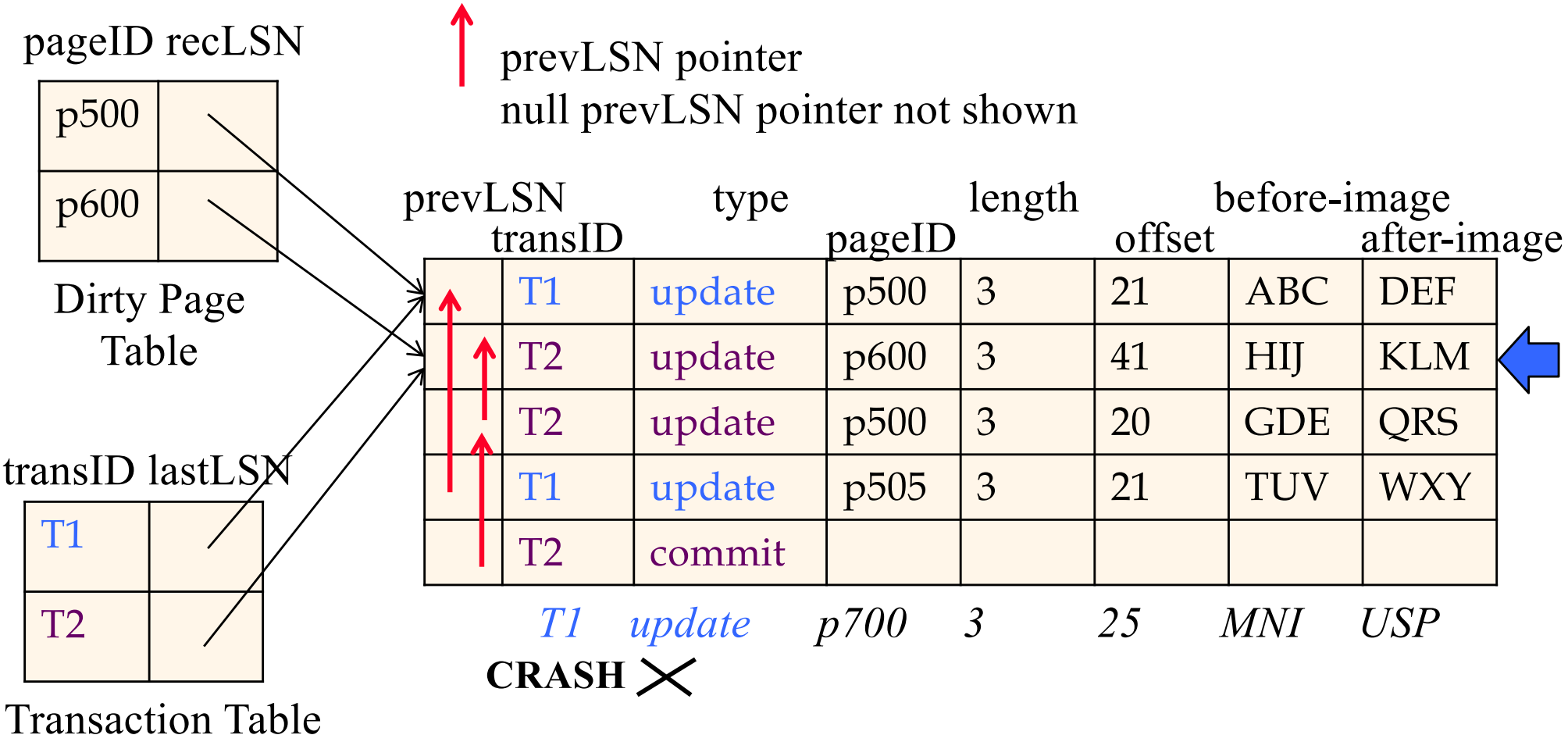
- p700 update not in the stable-storage log yet at crash time (just in the memory tail of the log)
- Assume analysis starts with an empty checkpoint record, and checkpoint was made just before this sequence of execution.

Example: Analysis Phase



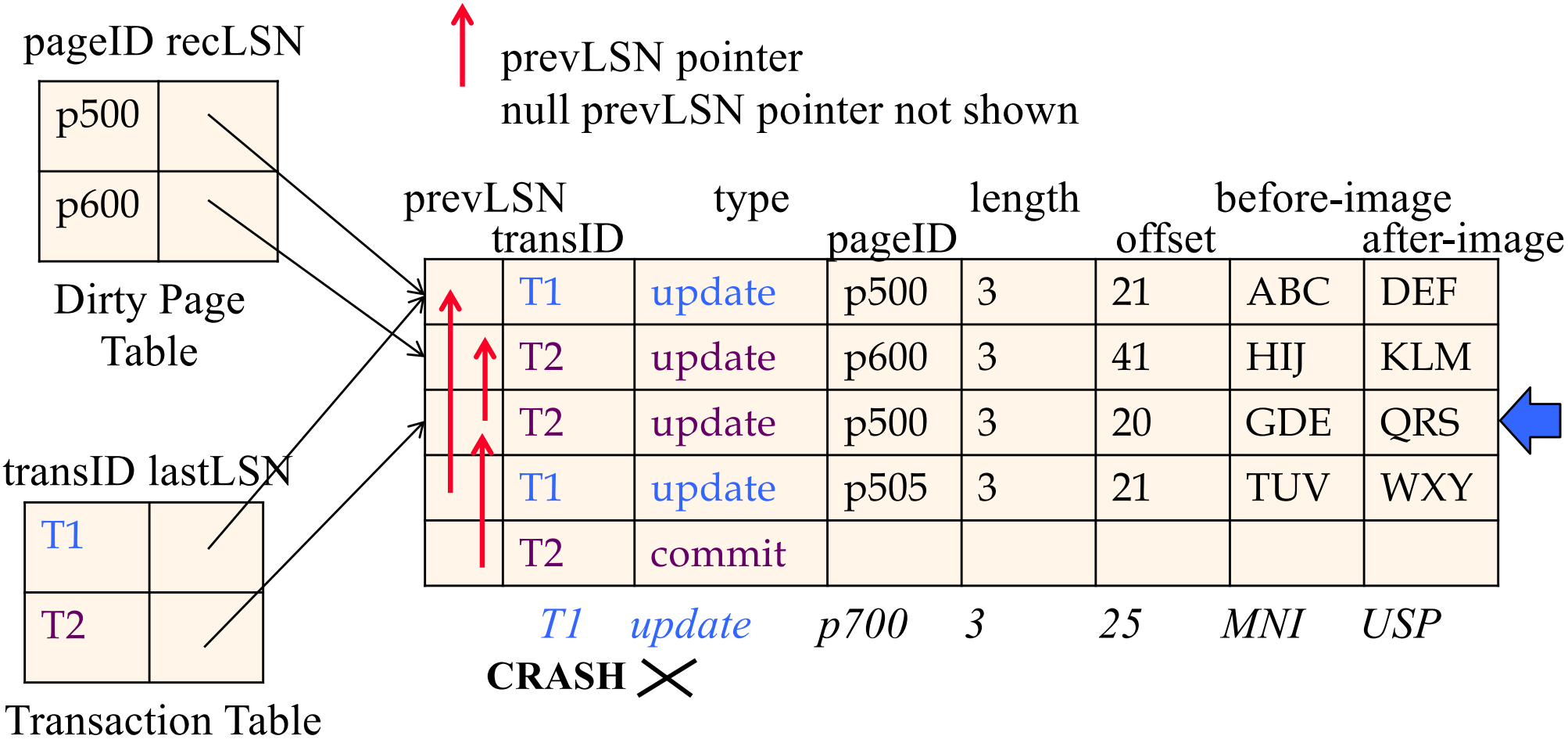
- p700 update not in the log yet at crash time (just in memory)
- Assume analysis starts with an empty checkpoint record, and checkpoint was made just before this sequence of execution.

Example: Analysis Phase (2)



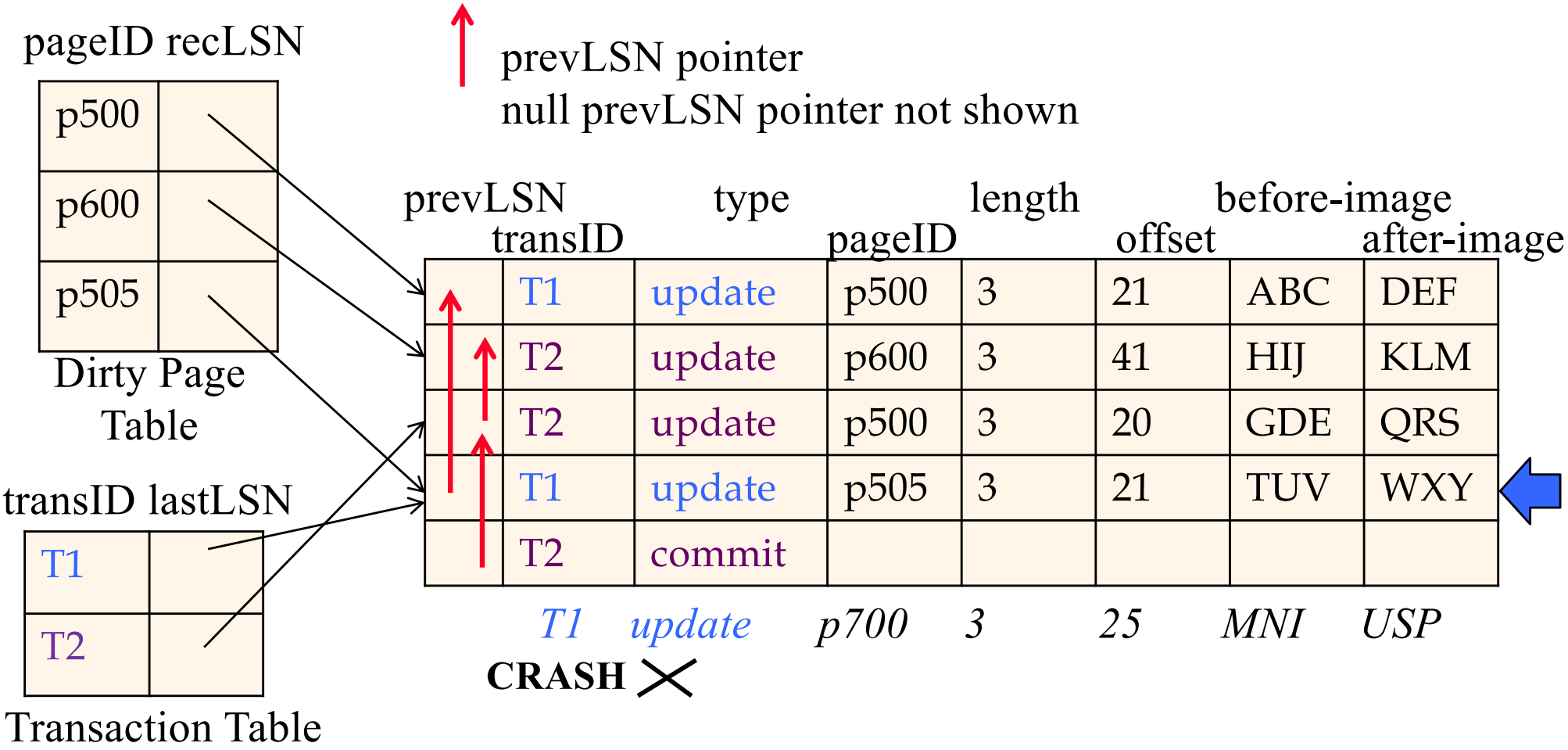
- p700 update not in the log yet at crash time (just in memory)
- Assume analysis starts with an empty checkpoint record, and checkpoint was made just before this sequence of execution.

Example: Analysis Phase (3)



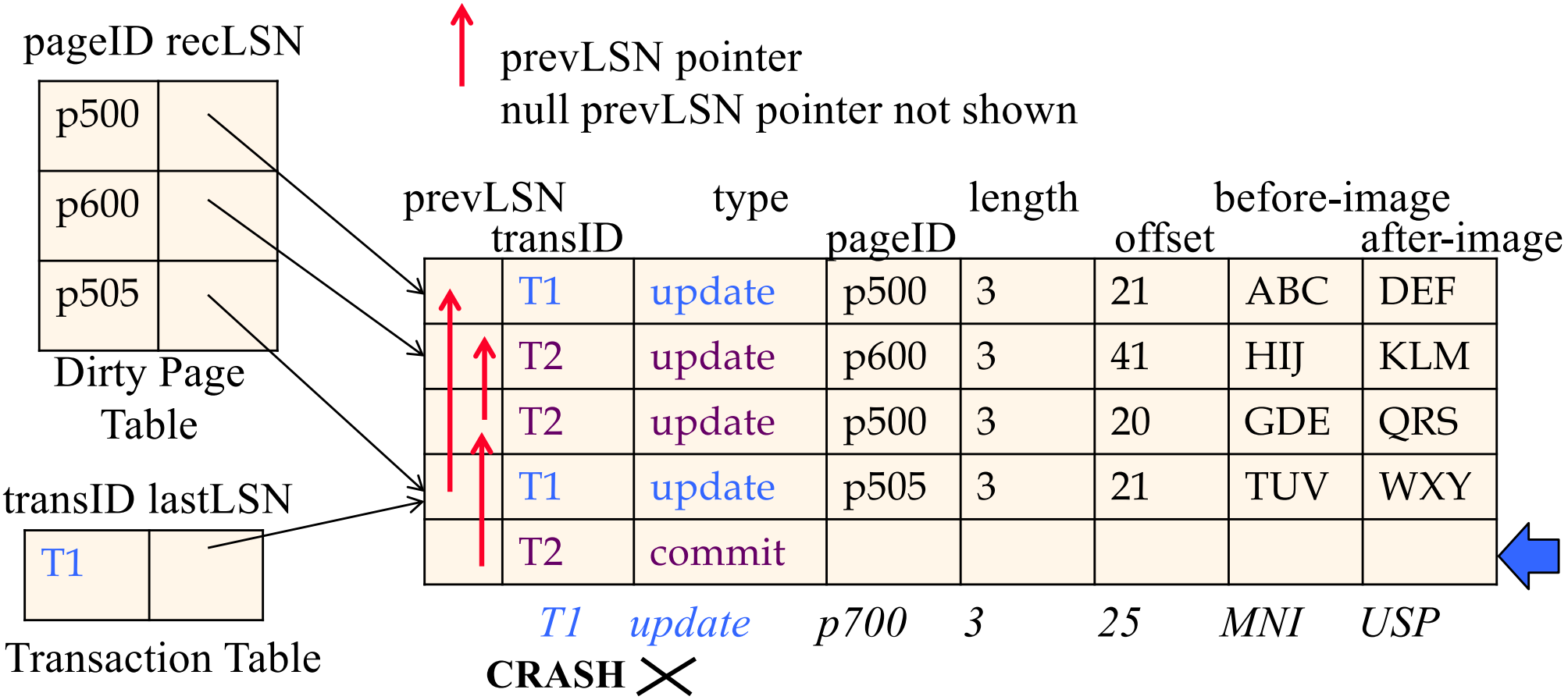
- p700 update not in the log yet at crash time (just in memory)
- Assume analysis starts with an empty checkpoint record, and checkpoint was made just before this sequence of execution.

Example: Analysis Phase (4)



- p700 update not in the log yet at crash time (just in memory)
- Assume analysis starts with an empty checkpoint record, and checkpoint was made just before this sequence of execution.

Example: Analysis Phase (5)



- Analysis phase is now finished. It recognized that only T1 was active in the system, with the lastLSN corresponding to the p505 update.
- The update to p700 is lost (not in the log), but not on the data, either!
- In general, not sure which updates of T2 are on disk. **Assume** only p600.

(2) *Recovery: The REDO Phase*

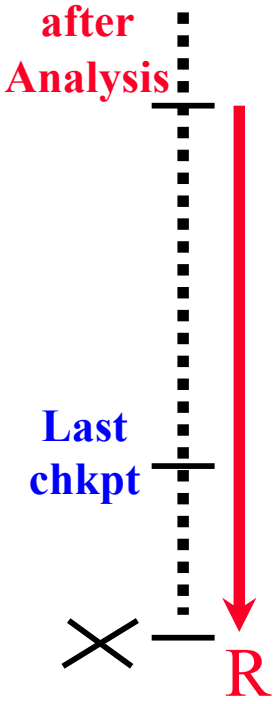
Repeat history to reconstruct *DB state at crash*:

- Reapply **all** updates, even those of aborted Xacts and redo CLRs.

a. Scan forward from the log record w/ smallest *recLSN* in DPT (oldest update that may not have been written to disk before crash)

b. For each (update or CLR) log record, REDO the action, unless:

1. It affects a page that is *not in the dirty page table*
   - No need to redo: this page has been flushed before crash!
2. Or the *page's recLSN is greater than the LSN of current record*
   - Just before recLSN, the page was clean (= disk image)
     → the first update that made this page dirty is after current record
     → the current update *has* been sent to disk.
3. Or the *pageLSN* (stored in the page) is >= *LSN of current record*.
   - The last update to this page that made it to disk, is the same as, or more recent than, the one for the current LSN.
   - This check is expensive (page on disk)! That's why we do it last.



Example: the REDO phase

- Smallest recLSN from DTP: 1st log record, p500.
- The page is dirty; its recLSN = 1st record LSN → read p500 from disk to find its pageLSN.

pageID recLSN

p500	
p600	
p505	

Dirty Page Table

transID lastLSN

T1	

Transaction Table

prevLSN	transID	type	pageID	length	offset	before-image	after-image
	T1	update	p500	3	21	ABC	DEF
	T2	update	p600	3	41	HIJ	KLM
	T2	update	p500	3	20	GDE	QRS
	T1	update	p505	3	21	TUV	WXY
	T2	commit					

CRASH

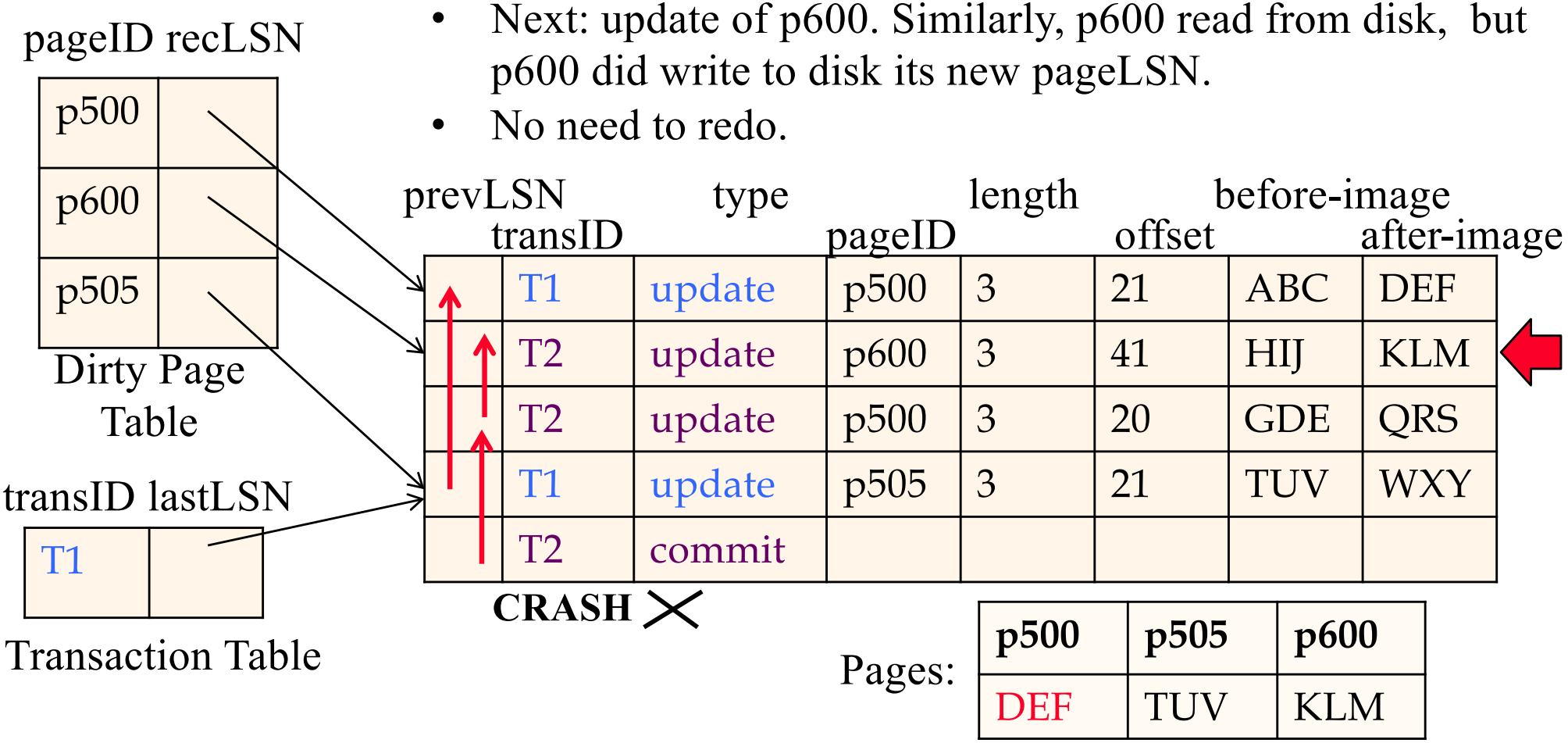
Pages:

p500	**p505**	**p600**
ABC	TUV	KLM

- We assumed only the update T2.p600 made it to disk before crash.
- Before the crash, the pageLSN of p500 in memory reflected update by T2.
- Thus, pageLSN of p500 < current LSN; the update is redone (ABC → DEF).

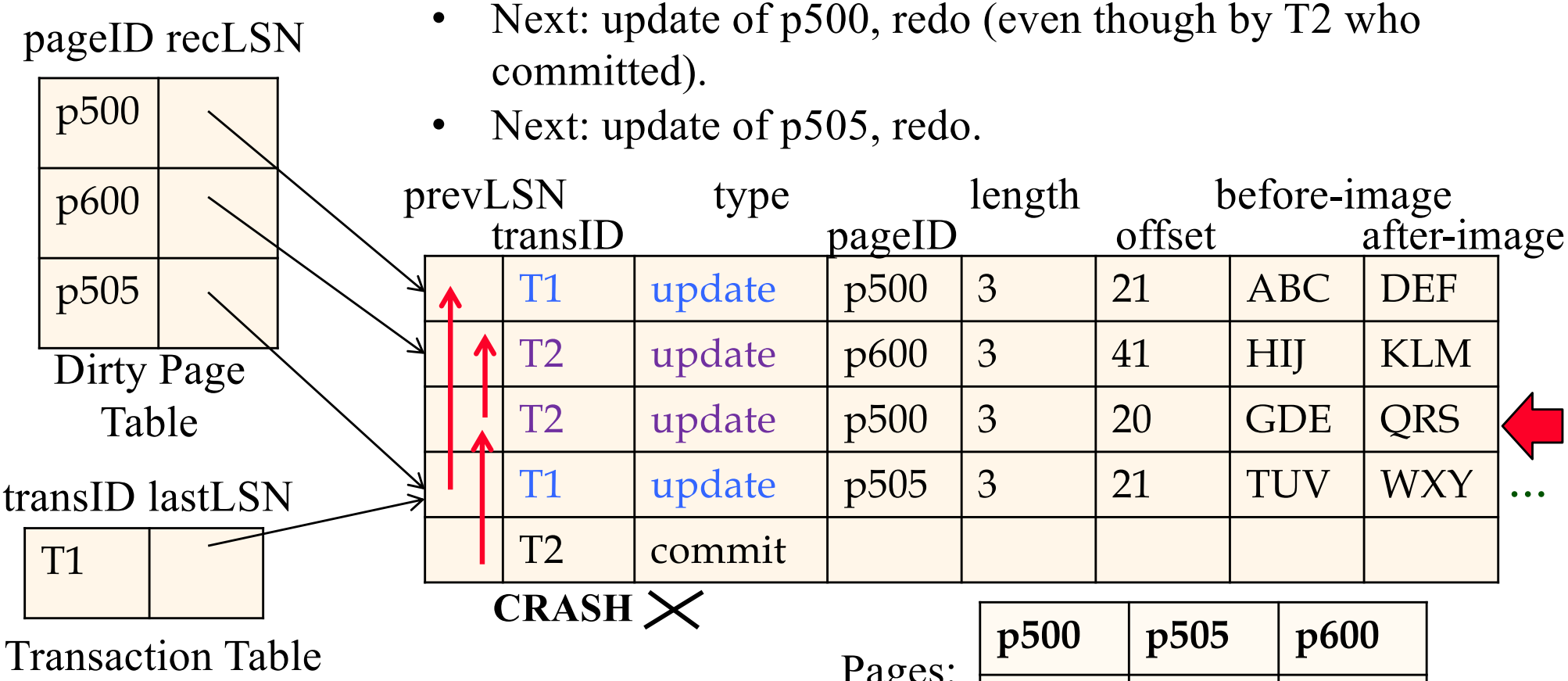
Example: the REDO phase (2)

- Next: update of p600. Similarly, p600 read from disk, but p600 did write to disk its new pageLSN.
- No need to redo.



Example: the REDO phase (3)

- Next: update of p500, redo (even though by T2 who committed).
- Next: update of p505, redo.



Pages:

p500	p505	p600
QRS	TUV	KLM

then

p500	p505	p600
QRS	WXY	KLM

(3) *Recovery: The UNDO Phase*

- Goal: undo the actions of all transactions that were active at the time of the crash, as identified by the Analysis phase.
  - These are called loser transactions.
  - At the end of the Analysis phase, we have the lastLSN of each loser transaction in the Xact table (see previous slide)
- Actions are undone in the reverse temporal order.

(3) *Recovery: The UNDO Algorithm*

ToUndo = { *l* | *l:* lastLSN of a "loser" xact}

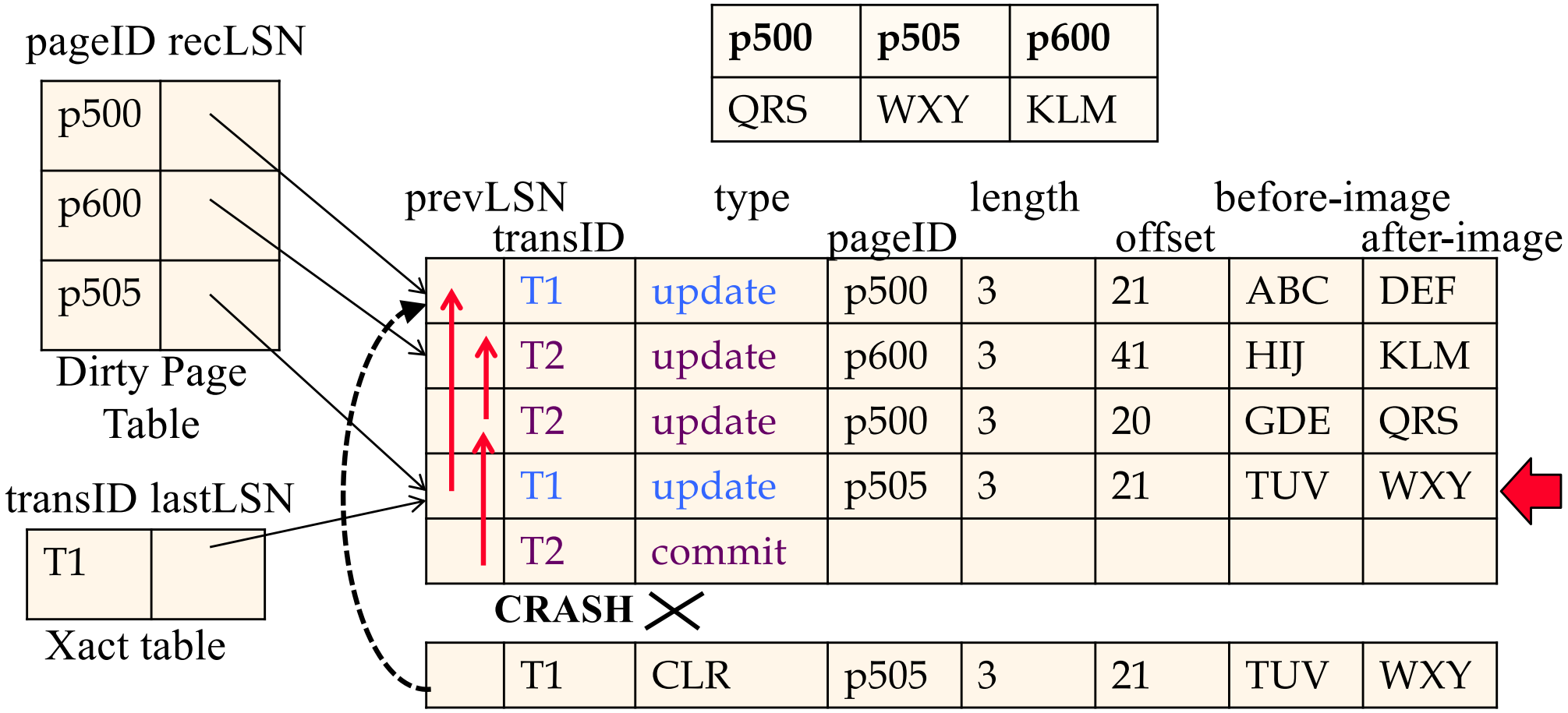
Repeat:

- Choose largest LSN among ToUndo.
- If this LSN is a CLR and undonextLSN==NULL
  - Write an End record for this Xact.
    (We have undone all effects of this Xact. We're done with it.)
- If this LSN is a CLR, and undonextLSN != NULL
  - Add undonextLSN to ToUndo

    (Make sure the previous record of this Xact will also be treated by the UNDO algo.)
- Else this LSN is an update. Undo the update, write a CLR, add prevLSN to ToUndo.

Until ToUndo is empty.

Example: the UNDO phase

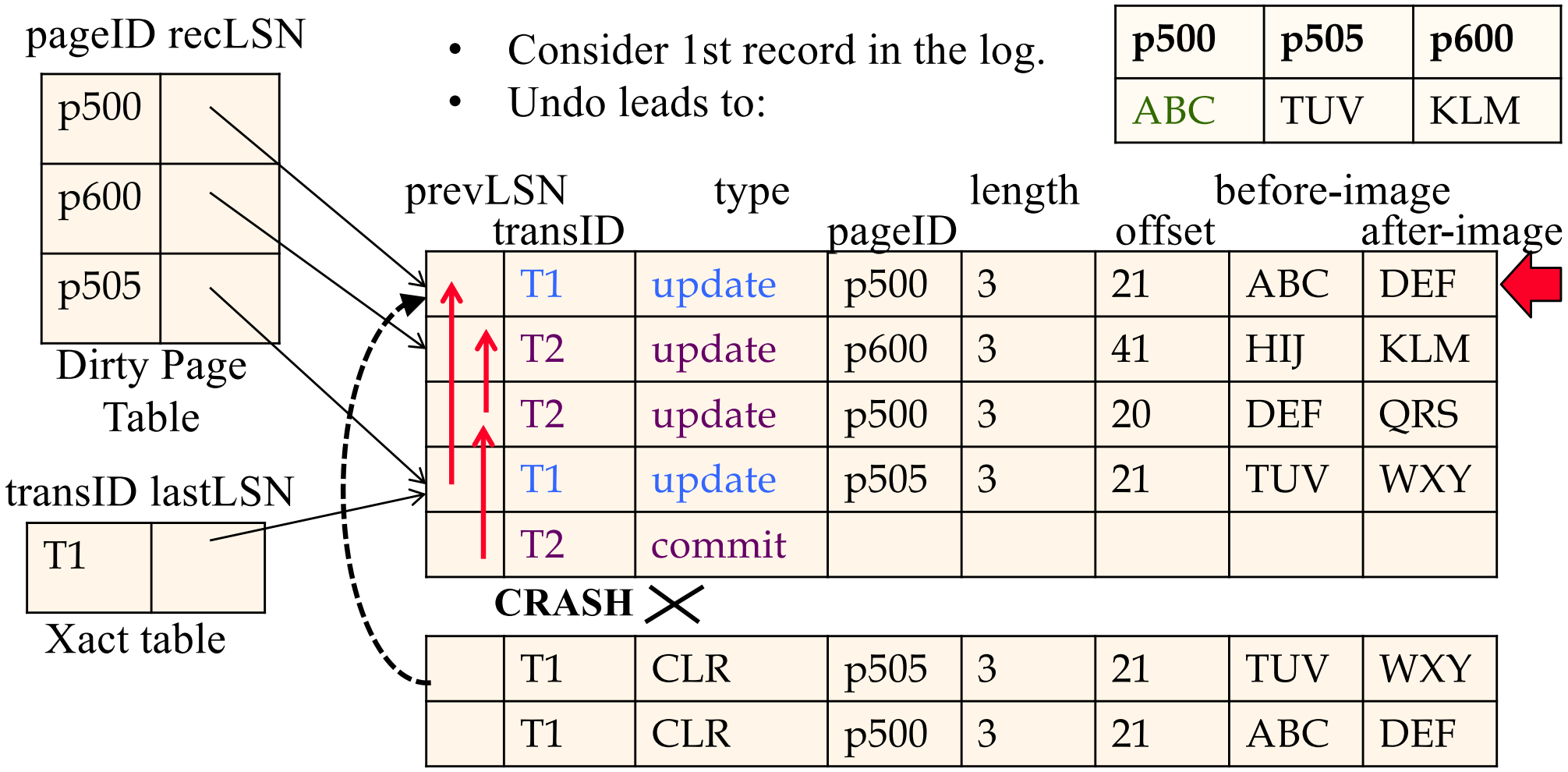


- Loser xact: T1; lastLSN: 4th log record.
- Undo leads to modifying p505:

p500	p505	p600
QRS	TUV	KLM

- Add CLR with undoNextLSN on 1st record.

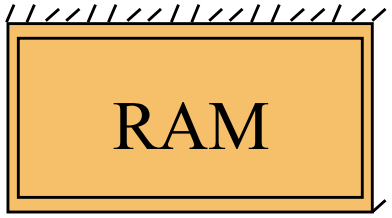
Example: the UNDO phase (2)



- This undo step lost a committed update of T2!
- T2 overwrote p500 while T1 still active.
- 2PL would not have allowed this scenario.

Recovery example also with Abort

Atomicity & *Durability*: none of T1, T2, T3 should have any effect on DB!



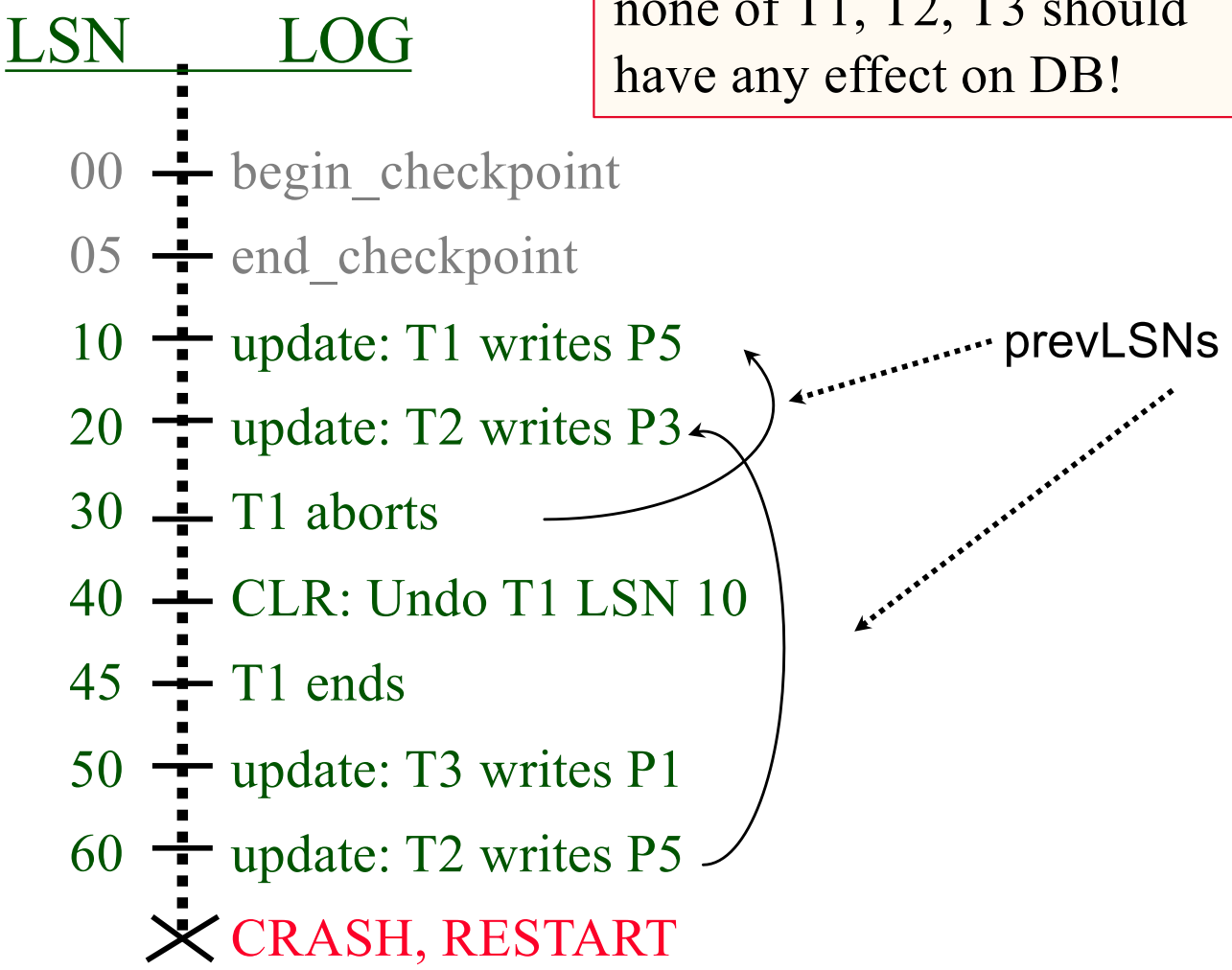
Xact Table (TT)

XID,
lastLSN,
status

Dirty Page Table (DPT)

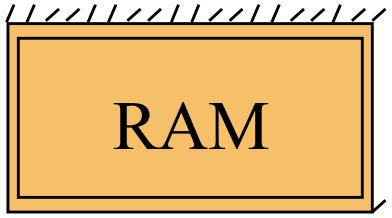
recLSN (earliest)

flushedLSN



Example of Recovery

Recovery algorithm:
ANALYSIS: TT & DPT?
REDO: T1, T2, T3
UNDO: T2, T3



Xact Table (TT)

lastLSN, status

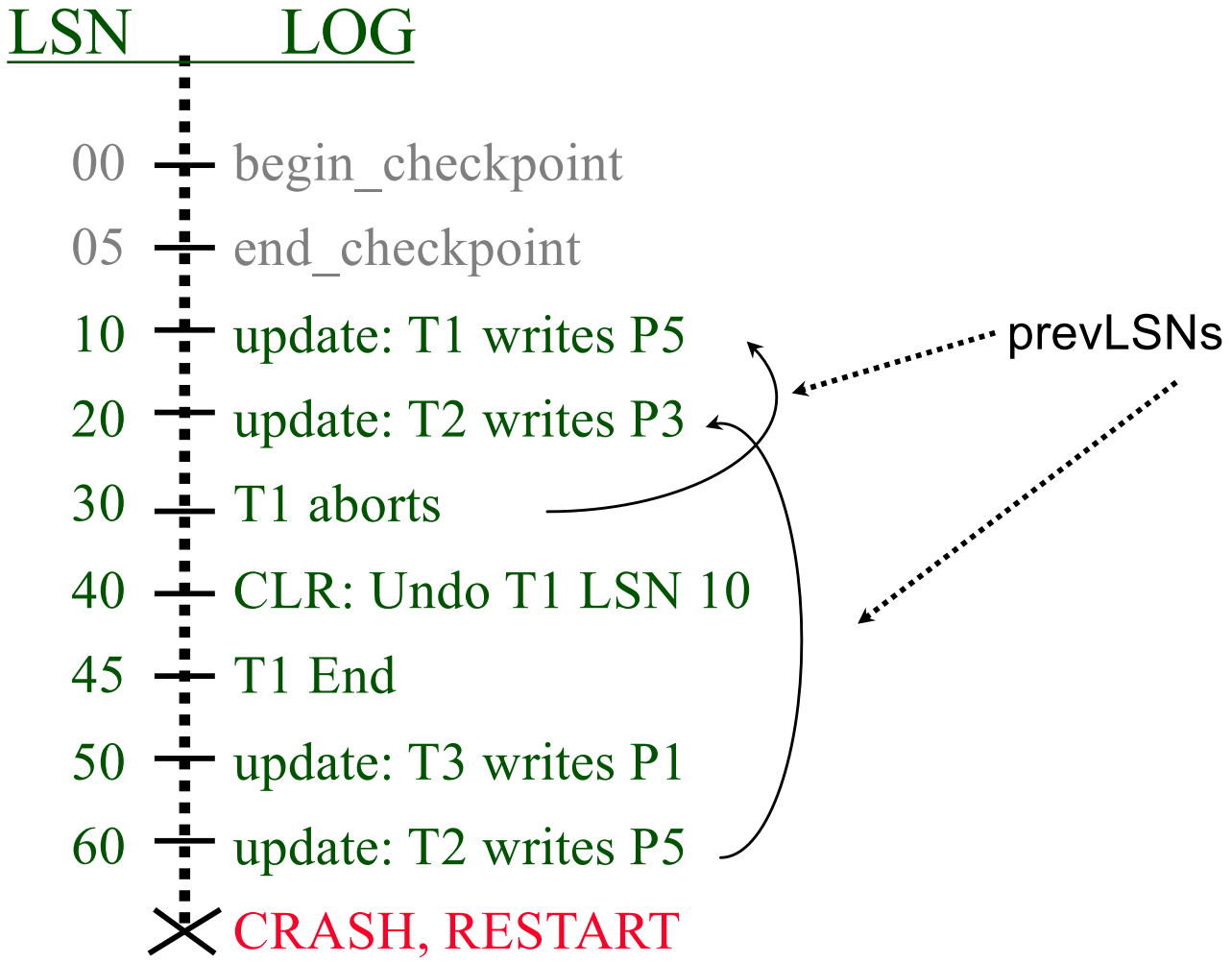
T2: 60, running
T3: 50, running

Dirty Page Table (DPT)

recLSN (earliest)

P5: 10
P3: 20
P1: 50

flushedLSN



Crash During Recovery

Atomicity & *Durability*: none of T1, T2, T3 should have any effect on DB!

Recovery algorithm:
ANALYSIS: TT & DPT?
REDO: T1, T2, T3
UNDO: T2

RAM

Xact Table (TT)

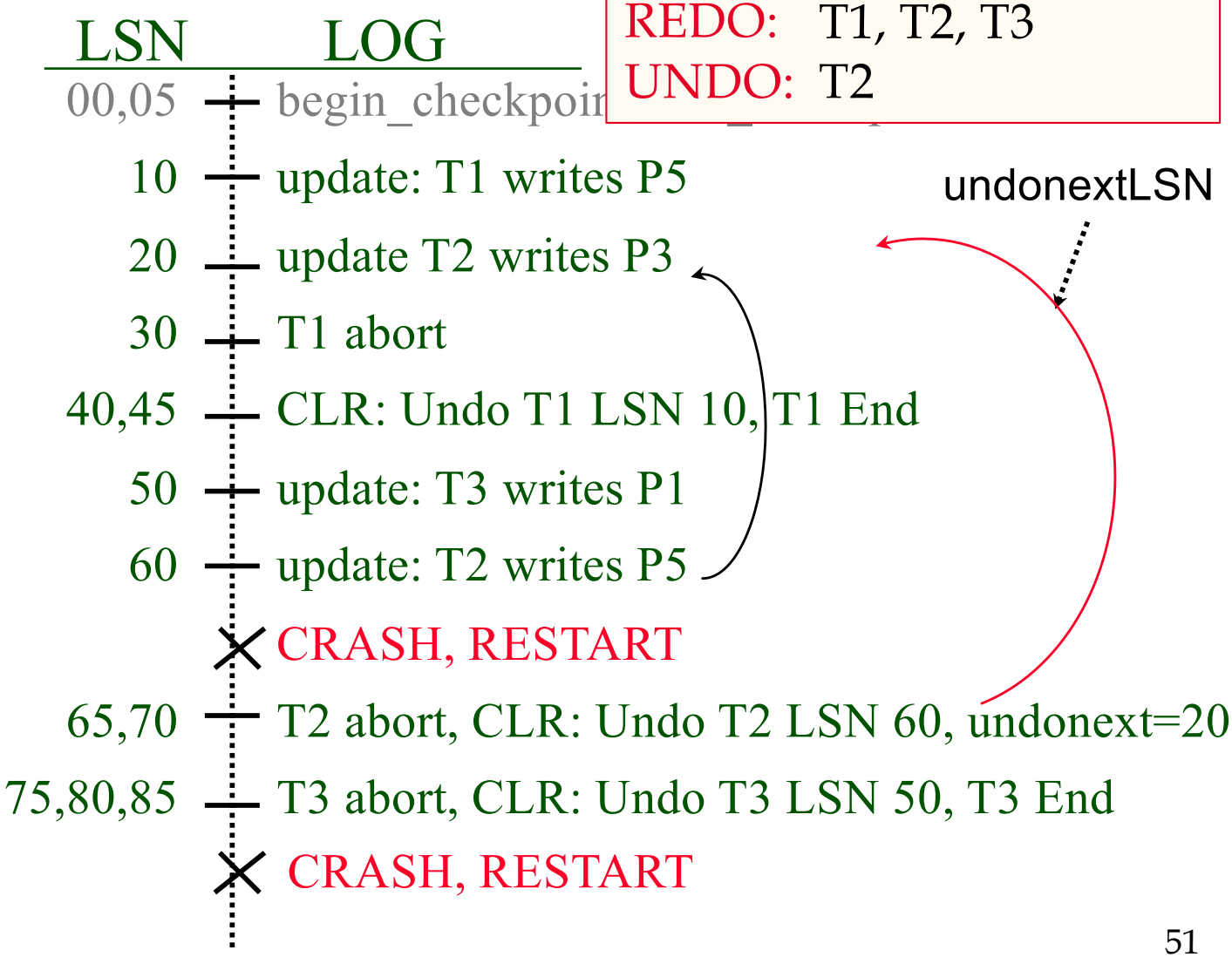
T2: 60, running
T3: 50, running

Dirty Page Table (DPT)

recLSN (earliest)

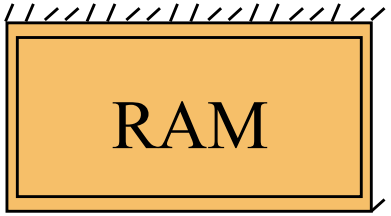
P5: 10
P3: 20
P1: 50

flushedLSN



Crash During Recovery

Recovery algorithm:
ANALYSIS: TT & DPT?
REDO: T2
UNDO: T2



Xact Table (TT)

T2: 60, running

Dirty Page Table (DPT)

recLSN (earliest)

P5: 10
P3: 20
P1: 50

flushedLSN

LSN	LOG
00,05	begin_checkpoint, end_checkpoint
10	update: T1 writes P5
20	update T2 writes P3
30	T1 abort
40,45	CLR: Undo T1 LSN 10, T1 End
50	update: T3 writes P1
60	update: T2 writes P5
	CRASH, RESTART
65,70	T2 abort, CLR: Undo T2 LSN 60, undonext=20
75,80,85	T3 abort, CLR: Undo T3 LSN 50, T3 End
	CRASH, RESTART
90,95	CLR: Undo T2 LSN 20, T2 end

undonextLSN

Main Principles behind ARIES

- Write-ahead logging (WAL)
  - Any change to an object is recorded in the log. The log is written to disk *before* the change to the object is written, or upon *commit*.
- Repeating history during REDO
  - On restart after a crash, repeat all actions before the crash, brings the system as close as possible to state that it was in before the crash.
  - Then undo the xacts still active at crash time.
- Logging changes during UNDO
  - Changes made to DB while undoing are logged to ensure such an action isn't repeated in the event of repeated restarts.
- (Fuzzy) Checkpointing to expedite recovery
  - An efficient way to create a snapshot of Xact Table & D.P.T.

Recovery in Relational Databases



NO!

Logging + ARIES recovery (w. checkpoints)
+ Replication

IBM DB2 Microsoft SQL Server ORACLE

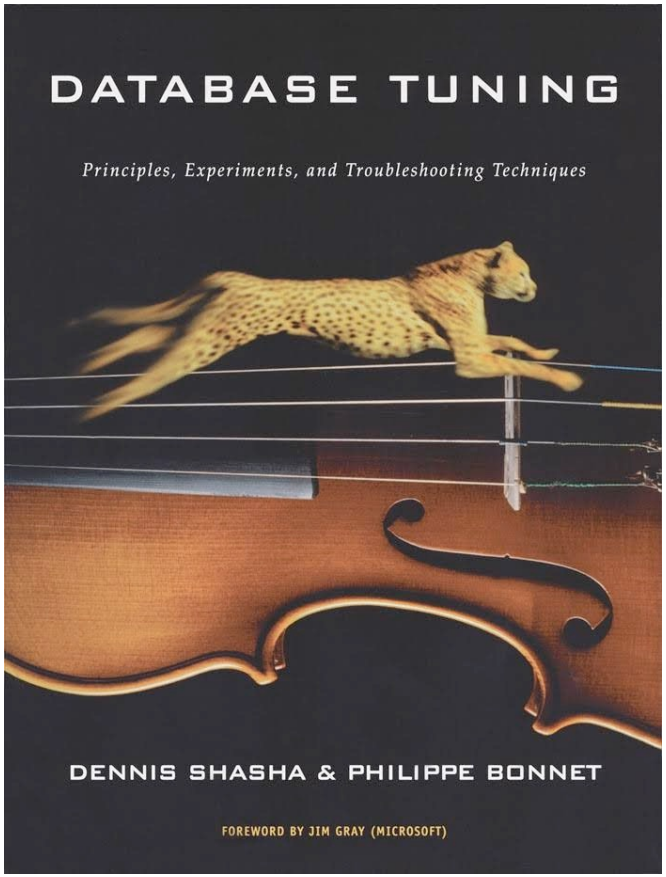
Additional Crash Issues

- What happens if system crashes during Analysis?
  - We don't lose anything.
- What if system crashes during REDO?
  - Just REDO it again later – the operation is idempotent.
- How do you limit the amount of work in REDO?
  - Flush asynchronously in the background.
  - Watch "hot spots": pages used very frequently (many user Xact in memory)
    - They are not expelled from memory (thus, not saved to disk) for a long time →
    - Their recLSN "gets old" → long history to redo.
- How do you limit the amount of work in UNDO?
  - Avoid long-running Xacts.

Performance overhead of logging

- Logging entails writing to disk.
- **Performance tip**: use a 2nd, separate disk for the log!
- The 2nd disk should be ideally on a different computer, in a different place

"We once consulted a company that said proudly, '*This is our log disk. The logs from all our databases go here!*' It was easy to be a hero that day. The bigger problem was to be nice about it."



Summary of Logging/Recovery

- Recovery Manager guarantees Atomicity & Durability.
- Use **WAL** to allow STEAL/NO-FORCE w/o sacrificing correctness.
- **LSNs** identify log records; linked into backwards chains per transaction (via **prevLSN**).
- **pageLSN** allows comparison of data page and log records.

Summary, Cont.

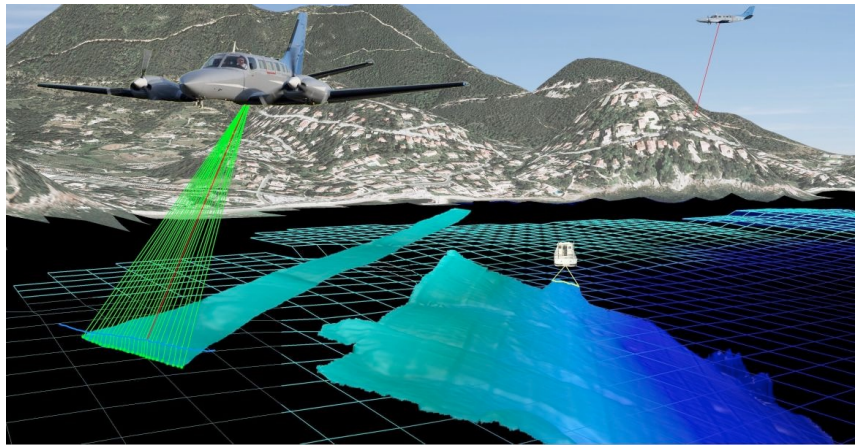
- Checkpointing: A quick way to limit the amount of log to scan on recovery.
- Recovery works in 3 phases:
  - Analysis: Forward from checkpoint.
  - Redo: Forward from oldest recLSN.
  - Undo: Backward from end to first LSN of oldest Xact alive at crash.
- Upon Undo, write CLRs.
- Redo "repeats history": simplifies the logic!

Outline

- Crash recovery overview
- Write-ahead logging (WAL)
- The ARIES recovery model
- More on durability: replication

Replication

- Consists of copying a relation, or a fragment of a relation, or a whole database on one or more sites.
  - To ensure durability even in the face of hardware (storage) destruction
    - Example: SHOM (Service Hydrographique et Océanographique de la Marine): back-up on tape



  - To increase availability during a software crash at one site (until it is back again)
  - To increase speed for queries that run on a data replica which is close to the query

Updating Distributed Data

- What is the impact of replication on transactions?
  - Updating one replica should lead to updating them all

- **Synchronous replication**: before the transaction commits, it synchronizes all the copies of the modified data
- **Asynchronous replication**: copies are only updated periodically, and a transaction that reads different copies of the same relation may see different values
  - In practice, one transaction usually works with one copy (the closest one)

Synchronous Replication

- Implemented by **voting**:
  - Each copy has a version number (highest is current)
  - A xact must write a majority of copies to modify an object, e.g., if there are 10 copies, at least 6
  - A xact must read enough copies to make sure at least one current copy is read; consider highest version
- Implemented by **read-any write-all**:
  - A xact can read from any copy.
  - A xact must write (thus, lock) all copies.
- In many scenarios, reads vastly outnumber writes. Thus, read-any write-all is preferred, as reading is faster.

Costs of Synchronous Replication

- Updates require locking on many copies.
- Thus, an xact needs to send lock requests to distant sites, wait to get the locks
  - During this time, the xact keeps the locks it currently has, limiting concurrency
- If a site or a connection fails, the xact cannot commit until all sites at which it has modified data have recovered and are reacheble.
- Committing the xact requires several additional messages

Asynchronous Replication

- Two flavors:
  - With a **primary site** (or master) and replicas on secondary sites (or slaves)
  - **Symetric** (peer-to-peer), not covered today.

- Primary-site asynchronous replication:
  - To write, always work with primary site.
  - Read is allowed from any secondary site.
  - Changes to the primary are propagated to the secondary sites.

Questions

